SC1006 Computer Organisation & Architecture
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Instruction Set Architecture & RISC

The contract between software and hardware: what the machine promises to do.

1.1 Why an ISA?

A computer translates intent into electrons: problem → algorithm → program → machine code → transistor switching. The boundary between software and hardware must be precise so a compiler targeting that boundary runs on any implementation of it. That contract is the **Instruction Set Architecture**.

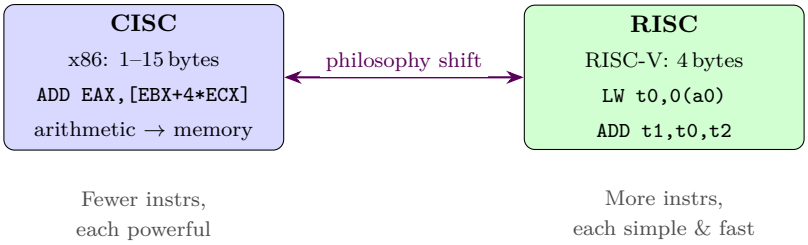
Definition 1.1 (ISA)**.** The programmer-visible state (registers, memory, program counter) plus the set of instructions, their binary formats, data types, addressing modes, and execution semantics.

Two machines sharing an ISA run the same binary; machines with different ISAs do not, even if both are "64-bit." Microarchitecture (pipeline depth, cache sizes, branch prediction) may differ wildly underneath the same ISA — that is the point: abstraction hides implementation. Changing the ISA breaks all existing software, so it is the most expensive interface to change; getting it clean pays dividends for decades.

1.2 CISC versus RISC: Two Philosophies

CISC (Complex Instruction Set Computer, e.g., x86-64): many instructions (≈ 1000), variable-length encoding (1–15 bytes), arithmetic may directly access memory, condition-code flags, microcoded string ops. Rationale circa 1970s: memory was tiny, compilers weak, so each instruction should do more work and save bytes.

RISC (Reduced Instruction Set Computer, e.g., RISC-V, ARM, MIPS): few simple instructions (≈ 50 base), fixed 4 byte encoding, load/store architecture, 32 general-purpose registers, 3–4 addressing modes, hardwired control. Rationale post-1980: make pipelining, caching, and compiler optimisation easy; let software compose power from simplicity rather than baking complexity into silicon.



RISC now dominates smartphones (ARM), is taking laptops/servers (Apple Silicon, ARM Neoverse, RISC-V), while x86 survives by cracking CISC instructions into RISC-like micro-ops inside the chip. A clean ISA lets transistors go to caches and predictors rather than decoders.

1.3 RISC-V: Our Running Example

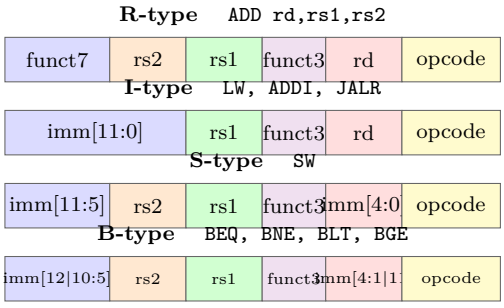
RISC-V is open, free, and modular. The base integer ISA is **RV32I**: 32 integer registers × 32 bit, a 32-bit program counter, and 2^{32} bytes of byte-addressable memory (little-endian). Extensions add multiply/divide (M), atomics (A), single/double float (F/D), compressed 2-byte encodings (C), vectors (V), etc.

Programmer-visible state:

- Registers `x0`–`x31`; `x0` ≡ 0 hardwired, `x1`=ra, `x2`=sp, `x3`=gp, `x4`=tp, `a0-a7` arguments/return, `t0-t6` temporaries, `s0-s11` saved.
- Program counter `PC`: byte address of next instruction; multiple of 4 without C extension.
- Memory: bytes, half-words (2 B), words (4 B); little-endian within each word.

1.3.1 Instruction Formats

Every RV32I instruction is 32 bits. Six formats (R, I, S, B, U, J) share the 7-bit opcode in bits [6:0] so decode can begin before the format is known; register fields are always in the same positions:



U-type (`LUI`, `AUIPC`: 20-bit upper immediate in [31:12]) and J-type (`JAL`: 20-bit jump offset shuffled) permute immediates similarly. Fixed opcode and aligned register fields let a single-cycle datapath read both source registers speculatively while decoding — saving mux delay.

1.3.2 Addressing Modes (only four!)

- **Register**: `ADD x3,x1,x2` — both operands in registers.
- **Immediate**: `ADDI x3,x1,42` — 12-bit signed constant embedded in the instruction.
- **Base+displacement**: `LW x3,8(x1)` — address = `x1`+8; word from memory into register.
- **PC-relative**: `BEQ x1,x2,off`, `JAL x1,off` — target = PC + sign-extended immediate.

RISC is *load/store*: only `LW/SW` (and variants `LH/LB/SH/SB`) touch memory; arithmetic never does. This regularity is precisely what makes pipelining (Ch. 4) feasible without heroic hardware.

1.4 From C to RISC-V Machine Code

```c
int sum(int *a, int n){
  int s=0;
  for(int i=0;i<n;i++) s+=a[i];
  return s;
}
```

RV32I assembly (`a0`=a, `a1`=n, return in `a0`; temporaries `t0`=s, `t1`=i):

```
sum:  mv   t0, zero
      mv   t1, zero            # i=0, s=0
loop: bge  t1, a1, done        # if i>=n exit
      slli t2, t1, 2            # t2 = i*4
      add  t2, a0, t2           # t2 = &a[i]
      lw   t3, 0(t2)            # t3 = a[i]
      add  t0, t0, t3           # s += a[i]
      addi t1, t1, 1            # i++
      jal  zero, loop
done: mv   a0, t0
      jalr zero, 0(ra)          # return
```

Each C construct maps to a short RISC sequence. Optimising compilers unroll loops and allocate registers to minimise loads/stores — jobs that a CISC decoder tries to do in hardware at higher power cost.

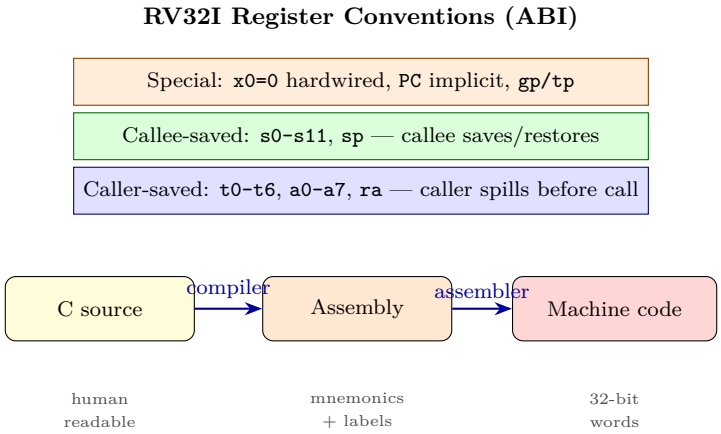
1.5 Endianness, Alignment, and the ABI

Endianness: RISC-V is little-endian. Storing `0x12345678` at 0x1000 via `SW` writes `0x78` to 0x1000, `0x56` to 0x1001, `0x34` to 0x1002, `0x12` to 0x1003. Network byte order is big-endian; forgetting to convert corrupts protocols.

Alignment: Word accesses expect addresses divisible by 4; half-words by 2. Misaligned access traps on strict RV32I. With the C extension, 16-bit compressed instructions may be half-word aligned.

Calling convention (ABI): `a0-a7` arguments/return, `t0-t6` caller-saved, `s0-s11` callee-saved, `ra` (`x1`)

return, `sp` (`x2`) stack pointer.



1.6 ISA Design Principles

A good ISA balances four pressures: (1) generality — express any computation; (2) efficiency — decode quickly and use registers well; (3) stability — binaries last decades; (4) extensibility — new features (crypto, vectors) fit without breaking old code. RISC-V scores well because its base is tiny and extensions are optional, so an embedded core implements RV32I while a server adds M/F/D/V without changing the base encoding.

1.7 Pseudo-Instructions and Assembler Sugar

The assembler provides *pseudo-instructions* that map to real ones: `mv rd,rs` ≡ `ADDI rd,rs,0`, `nop` ≡ `ADDI x0,x0,0`, `li rd,imm` expands to `LUI+ADDI` when needed, and `call offset` ≡ `AUIPC+JALR`. They do not affect the ISA — they are assembler conveniences that improve readability while assembling to canonical encodings.

1.8 Key Takeaways

A good ISA is a narrow waist: stable, simple, expressive for compilers, exposing enough for hardware to go fast. RISC-V achieves this with fixed-length formats and load/store discipline. Ch. 2 builds the datapath that executes these instructions; Ch. 3 adds the control that orchestrates it.

1.9 Exercises

Exercise 1.1. Encode `ADD x5,x6,x7` (opcode 0110011, funct3 000, funct7 0000000, rd 00101, rs1 00110, rs2 00111) as a 32-bit hex word.

Exercise 1.2. Why does RISC-V fix opcode in [6:0] and register fields in [19:15]/[24:20]/[11:7] across all formats? Explain speculative register read.

Exercise 1.3. With `a` in `t0`, `b` in `t1`, translate `if(a==b) a++; else b++;` into RV32I using at most one conditional branch.

Exercise 1.4. "CISC needs fewer instructions so code is smaller." Give two reasons RISC code-size overhead is acceptable in 2026.

Exercise 1.5. After `SW x5,0(x10)` with `x5`=0xAABBCCDD, `x10`=0x1000 (little-endian), list bytes at 0x1000–0x1003. Repeat for big-endian.

Chapter 2

Single-Cycle Datapath: ALU & Register File

From instruction bits to data flowing through adders, muxes, and memories.

2.1 From Zero: What a Single Instruction Must Do

A computer does only one thing: fetch an instruction from memory, figure out what it asks for, do the arithmetic or memory access, and store the result. Every RV32I instruction, no matter how different it looks in assembly, moves through the same five logical stages. Thinking in stages makes hardware systematic rather than ad-hoc:

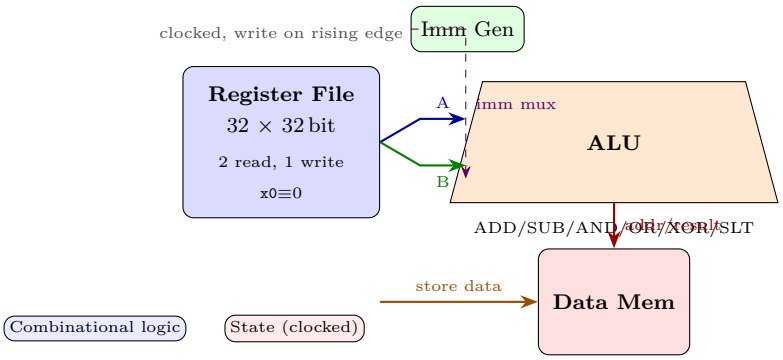
1. **IF** — Instruction Fetch: read the 32-bit word at address PC from instruction memory; concurrently compute PC+4 with a dedicated adder.
2. **ID** — Decode & Register Read: extract opcode, rd, rs1, rs2, funct3/7; read `rs1` and `rs2` from the register file; sign-extend the immediate.
3. **EX** — Execute: ALU operates on the two operands. For loads/stores this is address calculation (`rs1`+imm); for branches it is comparison; for R-type it is the actual operation.
4. **MEM** — Memory Access: only loads and stores touch data memory; other instructions simply pass the ALU result through.
5. **WB** — Write Back: write the result (ALU output or memory data) into `rd` in the register file, unless the instruction is a store or branch which has no destination.

A single-cycle machine does all five in one long clock period. The period must be long enough for the slowest instruction. That inefficiency is exactly what pipelining fixes in Ch. 4.

2.2 Building Blocks: The Datapath LEGO Set

The datapath is combinational logic plus state. State holds across cycles; combinational logic settles within a cycle. We need seven blocks:

- **PC register**: a 32-bit edge-triggered register holding the address of the current instruction. Updated every cycle to PC+4, branch target, or jump target.
- **Instruction memory**: combinational-read ROM/SRAM; address in, 32-bit instruction out. In the single-cycle abstraction we use separate instruction and data memories (Harvard view) to avoid a structural hazard; real chips use caches behind one memory.
- **Register file**: 32 × 32-bit, two read ports and one write port, with `x0` hardwired to zero.
- **ALU**: 32-bit combinational unit that can add, subtract, and, or, xor, shift, and set-less-than.
- **Data memory**: read/write SRAM for loads and stores; combinational read, synchronous write.
- **Immediate generator (ImmGen)**: pure wiring plus sign-extension that re-assembles the scattered immediate bits into a 32-bit signed value.
- **Adders and muxes**: a 32-bit adder for PC+4, another for branch target (PC+imm), and 2-to-1 muxes that steer operands and write-back data under control signals.

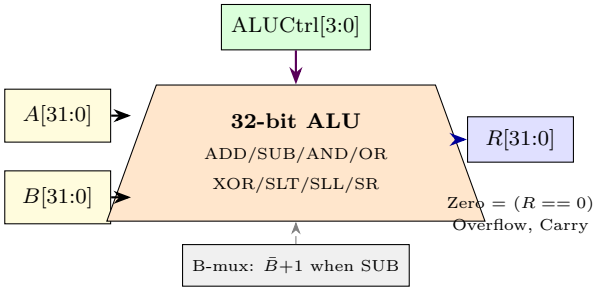


2.3 The Register File: 32 × 32 with 2R1W

Inside are 32 registers of 32 D flip-flops each, or an SRAM array. The interface is what matters: two asynchronous read ports and one synchronous write port. Give it `rs1` (bits [19:15]) and `rs2` [24:20]; within a fraction of a nanosecond the data appears at ReadData1 and ReadData2. Give it `rd` [11:7], WriteData, and `RegWrite`=1; at the next rising clock edge the register updates. Reads are combinational so ID and EX can happen in the same cycle; writes are edge-triggered so WB of one instruction does not corrupt ID of the same instruction. `x0` is special: reads always return 0 and writes are silently dropped. Hardware does this by forcing ReadData to 0 when address is 0, and by gating `RegWrite` with `(rd != 0)`. Without this, `x0` would drift and every `BEQ x0,x0` trick would break. Why two read ports? An R-type `ADD rd,rs1,rs2` needs both operands at once; a single-port file would need two cycles and double the CPI. The cost is an extra decoder and 32 extra 32-bit muxes, trivial compared to the ALU. The register file sits on the critical path: its read delay plus ALU plus memory determines cycle time, so it is built from fast, small SRAM with sense amplifiers, not dense DRAM.

2.4 The ALU: One Adder Does Both ADD and SUB

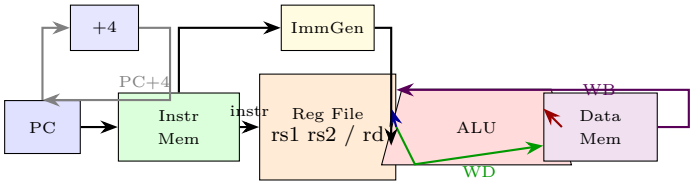
RV32I needs ADD (also for address), SUB (for `BEQ` via subtraction), AND, OR, XOR, SLL, SRL, SRA, SLT and SLTU. Rather than separate adder and subtractor, we share one 32-bit adder. Subtraction uses two's complement: $A - B = A + \bar{B} + 1$, where $\bar{B}$ is bitwise NOT and the +1 enters as carry-in.



So `ALUControl` does not switch between two adders; it inverts B and sets carry-in when it wants SUB or branch compare. The same trick gives `SLT`: compute $A - B$ and look at the sign bit, with overflow correction for signed comparison. Flags matter: `Zero` ($R == 0$) tells `BEQ/BNE` if two registers are equal; `Negative` and `Overflow` together give signed less-than. Shifts are a separate barrel shifter muxed in front of the adder output, selected by `ALUCtrl`. A 4-bit `ALUCtrl` thus covers all ten operations with room to spare, and the decoder in Ch. 3 maps `ALUOp`+funct to it.

2.5 Full Single-Cycle Datapath: Putting It Together

Wires now connect the blocks. Thick lines are 32-bit data; thin lines are control. Follow one instruction through the diagram below from left (PC) to right (write-back):



Walkthrough examples: `LW x5,8(x6)` fetches at PC, reads `x6`, ImmGen makes 8, ALU adds them to form address, data memory reads, and the WB mux selects memory data into `x5`. `SW x5,8(x6)` does the same address calc but writes ReadData2 (`x5`) into data memory and disables `RegWrite`. `ADD x5,x6,x7` uses both register operands, ALU does ADD, WB mux selects ALU result. `BEQ x5,x6,offset` subtracts via $A + \bar{B} + 1$, checks Zero, and muxes PC to branch target (PC+imm) when Zero=1; otherwise PC+4. `JAL x1,offset` writes PC+4 into `x1` and jumps to PC+imm unconditionally. Each mux is steered by one control bit; the full control table is in Ch. 3. The key insight is that data flows left-to-right, but only one path is active per instruction; muxes make a single datapath serve six instruction formats.

2.6 Immediate Generation: R/I/S/B/U/J and Why imm[0]=0

RISC-V immediates are scattered to keep opcode and register fields fixed. ImmGen is just wires plus sign-extension, selecting with the opcode. The regularity lets decode and register read overlap. The table below is worth memorising:

Format	Bits in instruction	Assembled 32-bit immediate	Range	Notes
R	none	0	—	no immediate
I	`instr[31:20]`	{{20{instr[31]}}, instr[31:20]}	−2048 to +2047	`ADDI, LW, JALR`
S	`[31:25]+[11:7]`	{{20{instr[31]}}, instr[31:25], instr[11:7]}	−2048 to +2047	`SW`: rs2 data separate
B	scrambled 12 bits	{19{instr[31]}, instr[31], instr[7], instr[30:25], instr[11:8], 0}	±4096	branch: imm[0]=0
U	`[31:12]`	{instr[31:12], 12'b0}	20-bit upper	`LUI/AUIPC`: low 12 zero
J	scrambled 20 bits	{11{instr[31]}, instr[31], instr[19:12], instr[20], instr[30:21], 0}	±1MB	`JAL`: imm[0]=0

Why is `imm[0]` always 0 for B and J? RISC-V instructions are half-word aligned (and word-aligned without compressed extension). A branch or jump offset measured in bytes is therefore always even; bit 0 would always be zero so it is omitted and hardwired to 0, gaining one extra bit of range. The scrambling of B and J (sign bit stays at `instr[31]`) looks messy but keeps the sign bit in one place for fast sign-extension and keeps the mux network shallow. Sign-extension replicates bit 31 across the upper bits; forgetting it makes negative offsets become huge positive ones, a classic lab bug. ImmGen also handles shift amounts: a 5-bit `shamt` from `rs2` or `imm[4:0]` for SLLI/SRLI/SRAI, zero-extended, not sign-extended.

2.7 Critical Path: What Sets the Clock

In a single-cycle machine the clock period must satisfy every instruction, so it is set by the longest combinational delay from PC clock edge back to the next PC or register setup:

$$T_{\text{clk}} \geq t_{\text{PC\_clk2Q}} + t_{\text{IMem}} + t_{\text{RF\_read}} + t_{\text{Mux}} + t_{\text{ALU}} + t_{\text{DMem}} + t_{\text{WB\_mux}} + t_{\text{RF\_setup}}$$

Branches also need the branch-adder and Zero comparison, but loads are still longer because they traverse both memories. Numerically, if $t_{\text{IM}} = 2\,\text{ns}$, $t_{\text{RF}} = 1\,\text{ns}$, $t_{\text{ALU}} = 2\,\text{ns}$, $t_{\text{DM}} = 2\,\text{ns}$, plus muxes and setup $\approx 1\,\text{ns}$, then $T_{\text{clk}} \approx 8\,\text{ns}$ or $125\,\text{MHz}$. An R-type instruction finishes after the ALU in $\approx 5\,\text{ns}$ but must wait the full $8\,\text{ns}$. A `BEQ` that fails still pays for the data-memory path even though it does not use it. This waste is the core argument for pipelining: overlap the stages so each cycle does one stage of one instruction, raising throughput by nearly 5× even though single-instruction latency stays similar. Ironically, the single-cycle datapath is the cleanest to understand precisely because everything is combinational between two clock edges, with no pipeline registers to reason about.

2.8 Step-by-Step Walkthrough: Six Instructions Through One Datapath

The same wires serve every instruction; only mux selects and enables change. Trace each: `ADD x5,x6,x7`: IF fetches at PC; ID reads `x6`,`x7`; EX ALU does $A + B$ with `ALUSrc`=0; MEM does nothing; WB mux picks ALU result into `x5`. RegWrite=1.

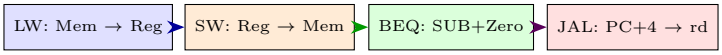
`ADDI x5,x6,42`: like ADD but `ALUSrc`=1 so ALU B comes from ImmGen (sign-extended 42) instead of `rs2`. Same WB as R-type.

`LW x5,8(x6)`: ImmGen gives 8; ALU adds `x6`+8 to form address; data memory reads 32 bits at that address; WB mux picks memory data (MemToReg=1) into `x5`. Needs both `MemRead` and `RegWrite`.

`SW x5,8(x6)`: address calc identical to LW, but ReadData2 (`x5`) is driven to DataMem write-data port and `MemWrite`=1; `RegWrite`=0 so no register is clobbered. The WB path is unused.

`BEQ x5,x6,offset`: ALU does $A + \bar{B} + 1$ (SUB) and Zero is tested; branch-adder computes PC+imm (with imm[0]=0); PC mux picks branch target only when Branch∧Zero=1. No memory, no WB. Getting the B-immediate scramble wrong flips forward/backward branches.

`JAL x1,offset`: ImmGen assembles J-immediate (imm[0]=0, ±1 MB); PC+4 is saved to `x1` via WB mux (a dedicated PC+4 path), and PC jumps to PC+imm. `JALR` is similar but target is `rs1`+imm with LSB cleared.



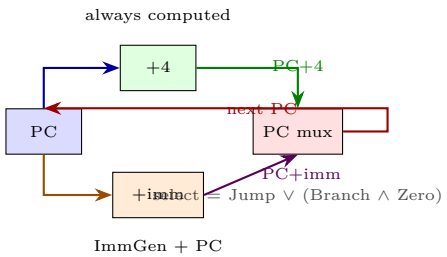
Only mux selects differ — datapath is shared

2.9 Timing: Why Single-Cycle Is Slow But Simple

Every signal must settle before the next rising edge. The PC register, instruction memory, register file read, ALUSrc mux, ALU (which itself is a 32-bit ripple or prefix adder), data memory, WB mux, and register setup all add. Loads dominate because they use both memories; R-type would be faster alone but must wait. Real numbers from a 45 nm library: PC 0.2 ns + IMem 1.8 ns + RF 0.9 ns + mux 0.2 ns + ALU 1.7 ns + DMem 2.0 ns + WB 0.2 ns + setup 0.3 ns ≈ 7.3 ns. Raising voltage or using faster SRAM shrinks this but never removes the waste: a `NOP` still pays for data memory it never uses. Pipelining solves it by overlapping, but its control and hazard logic only makes sense once this single-cycle baseline is crystal clear. A useful lab check: single-step the clock and probe each net; if DMem address wobbles after the ALU settles, the clock is too fast and `LW` will intermittently write garbage.

2.10 PC Logic: Choosing the Next Instruction

The PC is the only sequential state besides memories and registers. Next PC is a 3-way choice: PC+4 (default), PC+imm (branch taken or JAL), or rs1+imm with LSB cleared (JALR). Two adders run in parallel every cycle so the branch target is ready before we know if the branch is taken.



Control decides late: the Zero flag from the ALU (SUB) arrives just in time for the PC mux. If the clock were faster, Zero would still be settling when PC must latch, and branches would jump randomly. This is another reason loads set the period but branches stress the control timing. `JALR` reuses the ALU to compute rs1+imm, then clears bit 0 by AND with `0xFFFFFFFE`, a separate AND gate in the PC path.

2.11 Worked Example: B-type Immediate for −8 Bytes

Take `BEQ x5,x6,-8` meaning branch to PC−8. The byte offset −8 = `0xFFFFFFF8` = `0b11111111_11111000`. As a 13-bit signed value with imm[0]=0, this is `0b1_11111111_1100` (12:1 = all 1s except trailing 0). RISC-V scrambles it: instr[31]=1 (sign), instr[7]=0, instr[30:25]=`111111`, instr[11:8]=`1100`. Concatenated and sign-extended with trailing 0 gives `0xFFFFFFF8` again. A positive offset +8 would be `0b0_00000000_0100` with sign 0. Testing both +8 and −8 in simulation catches the classic bug where sign-extension is forgotten and −8 becomes +8184.

2.12 Why the Datapath Shape Is Not Arbitrary

Every design choice serves regularity. Fixed opcode position lets instruction memory and register file start together; scattered immediates keep register fields fixed at the cost of ImmGen wiring, but wiring is cheap and mux delay is expensive. Harvard memories in the diagram are a pedagogical fiction: real cores have one memory but split L1 I/D caches present the same dual-port illusion. The trap is to treat the diagram as literally separate chips; in layout, instruction and data paths interleave, but the logical separation lets us reason about one instruction at a time before hazards exist.

2.13 Common Pitfalls and Why This Matters

Three bugs catch every cohort: (1) forgetting `x0` gating, so `SW x0,0(sp)` mysteriously writes zero into `x0` and later reads back zero anyway, masking the error until `JALR` misbehaves; (2) treating B-type immediate as contiguous bits, producing off-by-2× branch targets that pass simple tests but fail backward branches; (3) sizing the clock for the ALU path and then watching `LW` intermittently corrupt because data memory had not settled before the write-back edge. All three vanish once the datapath is viewed as timing-constrained combinational logic between state elements, not just a block diagram.

2.14 Datapath Verification and Lab Bring-Up

On an FPGA, the single-cycle datapath is the first milestone. Connect the PC and memories to block RAM, wire the register file to flip-flops, and expose every control signal to LEDs or an ILA. A minimal test program:

Addr	Assembly	Hex (RV32I)
0x00	`ADDI x1,x0,10`	`0x00A00093`
0x04	`ADDI x2,x0,20`	`0x01400113`
0x08	`ADD x3,x1,x2`	`0x002081B3`
0x0C	`SW x3,0(x0)`	`0x00302023`
0x10	`LW x4,0(x0)`	`0x00002203`
0x14	`BEQ x3,x4,8`	`0x00418463`

Step one clock at a time. After cycle 1, `x1` must be 10; after cycle 3, `x3`=30; after cycle 4, DataMem[0]=30; after cycle 5, `x4`=30; after cycle 6, PC must have jumped to 0x1C if `x3==x4`, otherwise stayed at 0x18. Probing `Zero` and `ALUCtrl` on the ILA catches the SUB-vs-ADD bug instantly. Real bring-up also checks `x0`: after `ADDI x0,x0,99`, `x0` must remain 0 and `RegWrite` gating is verified.

2.15 Harvard Illusion and Why Two Memories Are Drawn

The diagram shows separate instruction and data memories so IF and MEM do not collide in one cycle. In a real chip there is one DRAM behind caches, and the Harvard view is created by split L1 I/D caches presenting two ports. For the single-cycle abstraction this detail is hidden; the invariant is that PC→IMem and ALU→DMem are independent combinational paths in the same clock. When we pipeline, the split matters: IF and MEM may access cache in the same cycle, so a unified cache would be a structural hazard and we need split L1.

2.16 Exam Checklist: Datapath

For any datapath question follow this order: (1) list stages IF/ID/EX/MEM/WB and what each block does; (2) state register-file ports (2R1W, `x0`=0, async read, sync write); (3) explain SUB as $A + \bar{B} + 1$ and which `ALUCtrl` selects it; (4) walk one instruction through the full diagram naming each mux select; (5) write the ImmGen row for its format and note `imm[0]`=0 for B/J (half-word alignment); (6) name the critical path (PC→IMem→RF→ALU→DMem→WB→RF setup) and which instruction sets it (`LW`); (7) justify why single-cycle period wastes time for R-type and motivates pipelining.

2.17 Data Memory: Why Loads Set the Period

Data memory is the other slow block. Reads are combinational (address in, data out within t_{DMem}); writes are synchronous (data and address must be stable before the clock edge, write-enable gates the clock). For `LW`, the ALU result must settle at the address port, then memory reads, then the WB mux selects it, all before register setup. For `SW`, ReadData2 must be stable at the write-data port while the address is also stable; the write itself happens at the edge so it does not affect the current cycle's WB. A common error is to make data memory synchronous-read: then `LW` would need an extra cycle and the single-cycle abstraction breaks.

2.18 Shifts and SLT: Beyond the Adder

SRL/SRA/SLL are not done by the adder but by a barrel shifter: a tree of 2-to-1 muxes that shifts by 1,2,4,8,16 in stages, selected by `shamt`[4:0]. SLT is subtle: `SLT rd,rs1,rs2` sets `rd`=1 if `rs1`<`rs2` signed, else 0. Hardware does SUB and looks at sign and overflow: result is `Negative`⊕`Overflow`. `SLTU` is unsigned, so it uses Carry instead. Both reuse the adder path; the ALU mux just selects the 1-bit result zero-extended to 32 bits instead of the 32-bit sum.

2.19 Two More Drills: AUIPC and JALR

`AUIPC x5,0x12345` forms `0x12345`≪12 plus PC and writes it to `x5`. Datapath: ImmGen makes `0x12345000`, ALU adds PC+imm (a separate PC→ALU mux when `ALUSrc`=1 and a PC-select bit is set), WB picks ALU. `JALR x1,0(x5)` computes `x5`+0 via ALU, clears bit 0, jumps there, and writes PC+4 to `x1`. The LSB clear is AND with `0xFFFFFFFE`; forgetting it misaligns the fetch and traps. Both instructions show why ImmGen's U-type (low 12 zero) and I-type share no hardware except sign-extension.

2.20 Reading the Waveform: What to Watch on the ILA

A correctly clocked single-cycle core shows a clean staircase: PC increments by 4 each cycle except when a taken branch or jump replaces it. Register write data appears one memory delay after the address. On the waveform, `LW`'s write-back pulse is late (after DMem), while `ADD`'s is earlier (after ALU). If `BEQ` is taken, the next PC is PC+imm, not PC+4; if not taken, PC+4. Capturing `Zero` and `Branch` at the rising edge proves the comparison landed in time. A failing setup shows as `Zero` still transitioning when PC latches, and the next fetch is from a phantom address.

2.21 Single-Cycle vs Multi-Cycle vs Pipeline

Single-cycle is the simplest abstraction: one long period, no pipeline registers. Multi-cycle reuses one ALU and one memory across states (fetch, decode, execute, memory, write-back) with a state machine, saving area but needing control sequencing. Pipeline keeps the single-cycle datapath blocks but inserts registers between stages so five instructions overlap. Learning single-cycle first makes the other two trivial: multi-cycle is time-division of the same blocks, pipeline is space-division. All three share ImmGen, register file, and ALU; only control and registers change.

2.22 Synthesis Reality: What Maps to LUTs

On an FPGA, each combinational block becomes LUTs. The 32-bit adder is 32 LUTs with carry chain; the register file is 32×32 flip-flops plus two 32:1 muxes (each 160 LUTs); ImmGen is wiring plus a 12-bit sign-extender (a few LUTs). The whole single-cycle core fits in under 800 LUTs on an Artix-7, leaving room for the CSR file. Timing reports show the same critical path as the ASIC estimate: IMem (block RAM Tco 1.2 ns) +

RF (0.6 ns) + ALU (1.1 ns) + DMem (1.4 ns) dominates. Closing timing means adding a pipeline register, not faster LUTs.

2.23 Why RISC-V Chose This Immediate Layout

MIPS kept immediates contiguous; RISC-V scatters them so opcode and register fields never move. The trade is one ImmGen mux versus a wider decode mux that would be on the critical path. Since wires are cheaper than mux delay, RISC-V wins. The choice also makes compressed (16-bit) extension easier: its immediates reuse the same scatter pattern. This is a lesson in ISA design: optimise the common hardware (decode) even if the rare instruction (branch) pays with wiring.

2.24 Forward-Look: What Changes When We Pipeline

The single-cycle datapath inserts no registers between stages. To pipeline, cut the wires between IF/ID, ID/EX, EX/MEM, MEM/WB and insert edge-triggered pipeline registers that hold instruction bits, control signals, and data for one extra cycle. The datapath blocks themselves do not change: the same ALU, same ImmGen, same memories. Only the clock period shrinks from T_{LW} to $\max(t_{\text{IM}}, t_{\text{RF}} + t_{\text{ALU}}, t_{\text{DM}})$ plus register overhead. That shrinks from $\sim$8 ns to $\sim$2 ns, at the cost of hazards that Ch. 4 solves with forwarding and stalling.

Single-cycle control is trivial (combinational) but the datapath itself is the foundation: every later optimisation (multi-cycle, pipeline, superscalar) is a scheduling of these same blocks.

2.25 Bonus Drill: Cycle-by-Cycle Table

Fill this table for three cycles executing `ADDI x1,x0,10; ADD x2,x1,x1; SW x2,0(x0)`:

Cycle	PC	Instruction	Key datapath action	RF change
1	0x00	`ADDI x1,x0,10`	ImmGen 10 → ALU ADD → WB	`x1`=10
2	0x04	`ADD x2,x1,x1`	`x1`+`x1` → ALU → WB	`x2`=20
3	0x08	`SW x2,0(x0)`	`x0`+0 → ALU addr; `x2` → DMem	Mem[0]=20

If any cycle shows Mem[0] still 0, the store's `MemWrite` was not asserted; if `x2` is 10, the second read port was wired to `rd` instead of `rs2`.

2.26 Glossary and Signals You Must Know

Quick reference: PC (program counter), IMem/DMem (instr/data memory), RF (register file), ImmGen (immediate generator), ALUCtrl/ALUOp (ALU control), MemRead/MemWrite (data enables), RegWrite (register write), MemToReg (WB mux), ALUSrc (ALU B mux), Branch/Jump (PC mux), Zero/Negative/Overflow (ALU flags), `shamt` (shift amount).

2.27 Exercises

Exercise 2.1. For `LW x5,8(x6)`, trace data flow through the full datapath. Which muxes select which inputs at each stage, and what does data memory do?

Exercise 2.2. For `AUIPC x5,0x12345` at PC 0x1000, what ImmGen immediate is produced and what value reaches `x5` after WB?

Exercise 2.3. Why must the register file have two read ports? Could a single-port file with two cycles work? What would it cost in CPI and clock period?

Exercise 2.4. The ALU does SUB by inverting B and setting carry-in to 1. Show how `BEQ` uses this plus the Zero flag to test equality without a separate comparator; what ALUCtrl does it use?

Exercise 2.5. If $t_{\mathrm{IM}} = 2\,\mathrm{ns}$, $t_{\mathrm{RF}} = 1\,\mathrm{ns}$, $t_{\mathrm{ALU}} = 2\,\mathrm{ns}$, $t_{\mathrm{DM}} = 2\,\mathrm{ns}$, estimate T_{clk} for single-cycle. Which instruction determines it and why? How much would R-type alone need?

Exercise 2.6. Design ImmGen for B-type: list which instruction bits map to `imm[12:1]`, explain why `imm[0]` is always 0, and compute the immediate for branch offset -8 bytes showing all scrambled fields.

Chapter 3

Control Unit: Orchestrating the Datapath

If the datapath is the body, control is the nervous system: every mux and enable has a signal.

3.1 What Control Does

The datapath has many choices each cycle: which register to read/write, ALU operation, whether to access memory, where the next PC comes from. The **control unit** examines the instruction word (opcode, funct3, funct7) and generates the select/enable signals that configure the datapath. In a single-cycle machine all decisions are combinational; in a pipelined machine they flow through pipeline registers (Ch. 4).

3.2 Control Signals Inventory

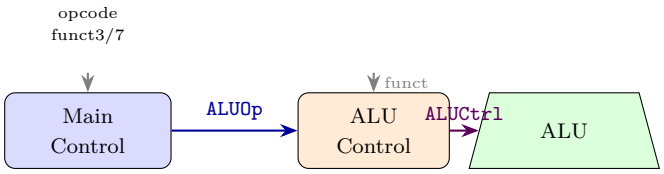
For our RV32I single-cycle datapath:

Signal	Width	Meaning
`RegWrite`	1	write register file when 1
`MemRead/MemWrite`	1 each	data-memory read/write
`MemToReg`	1	write-back source: 0=ALU, 1=memory
`ALUSrc`	1	ALU input B: 0=register, 1=immediate
`ALUOp/ALUCtrl`	2 / 4	selects ALU operation
`Branch`, `Jump`	1 each	PC source selection
`ImmSrc`	2	which immediate format

3.3 ALU Control: From funct to Operation

The main control emits a 2-bit `ALUOp`; the ALU decoder combines it with funct fields to produce 4-bit `ALUCtrl`:

ALUOp	funct	ALUCtrl / operation
00	—	0010 = ADD (address / ADDI)
01	—	0110 = SUB (branch compare)
10	funct3=000, funct7=0	0010 = ADD
10	funct3=000, funct7=0100000	0110 = SUB
10	funct3=111	0000 = AND
10	funct3=110	0001 = OR
10	funct3=100	0011 = XOR
10	funct3=010	0111 = SLT



3.4 Main Control Truth Table

Each opcode maps to a control-word (combinational). Example rows:

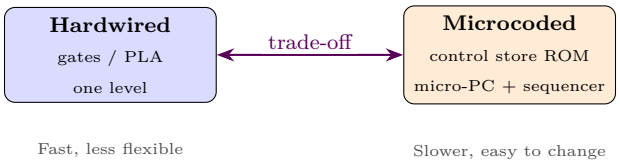
Instr	ALUOp	ALUSrc	MemRead	MemWrite	RegWrite	MemToReg	Branch	Jump
R-type (0110011)	10	0	0	0	1	0	0	0
LW (0000011)	00	1	1	0	1	1	0	0
SW (0100011)	00	1	0	1	0	—	0	0
BEQ (1100011)	01	0	0	0	0	—	1	0
JAL (1101111)	—	—	0	0	1	—	0	1

"--" means don't-care; logic minimisation (Karnaugh / espresso) exploits them to reduce gate count. Branches use SUB and check the Zero flag; JAL writes PC+4 to rd and jumps.

3.5 Hardwired versus Microcoded Control

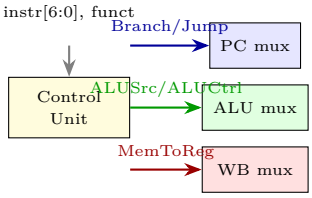
Hardwired: control truth table synthesised directly into gates (PLA / random logic). Fast, used in RISC single-cycle and pipelined designs.

Microcoded: instructions as sequences of micro-ops stored in a control ROM; micro-PC steps through them (CISC style). More regular but slower; modern x86 decodes to hardwired micro-ops internally — a hybrid.



3.6 Single-Cycle Control Datapath Integration

All control outputs fan out to muxes and enables. The PC logic is itself controlled: next PC is PC+4 normally, branch target if `Branch`∧`Zero`, or jump target if `Jump`. The beauty of RISC: because formats are regular, main control is a small truth table (≈ 7 opcodes × 8 signals) and easily pipelined.



3.7 Exceptions and Interrupts (Preview)

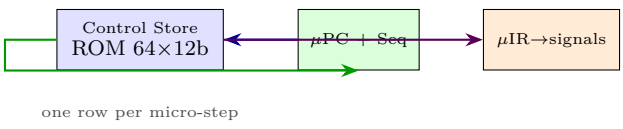
Real control also handles traps: illegal opcode, misaligned access, `ECALL`/`EBREAK`, and external interrupts. On trap, control saves PC to `mepc`, cause to `mcause`, and jumps to a handler address in `mtvec` (RISC-V privileged spec). Pipelined control must flush in-flight instructions and suppress their writes — a control hazard handled by injecting bubbles.

3.8 Hardwired Control in Detail

Hardwired means a PLA or random gates directly implement the truth table. Inputs are opcode[6:0], funct3, funct7[5]; outputs are the 8 control lines. Synthesis tools minimise with espresso, exploiting don't-cares to merge product terms. The result is two-level logic (AND-OR) or multi-level gates, settling in under 0.5 ns in 45 nm. Changing the ISA means re-synthesising, but RISC's small table makes this trivial. This is why every high-performance RISC core is hardwired.

3.9 Microcoded Control in Detail

Microcoded machines store control words in a ROM. A sequencer steps through micro-ops: fetch, decode, execute, memory, write-back each become one or more micro-cycles. The micro-PC increments or branches on opcode. Adding a new instruction means writing a new micro-routine, not rewiring gates. Cost is time: each micro-step is a clock, so an R-type that was one cycle now takes 4. CISC used this when transistors were scarce; modern x86 still microcodes complex ops (e.g., `REP MOVS`) but cracks simple ops into hardwired micro-ops.



3.10 Trap Handling: When Control Must Abort

Illegal opcode (no matching truth-table row), misaligned load/store (address[1:0]≠0 for LW), `ECALL/EBREAK`, and external interrupts all trap. Hardware does atomically: save PC to `mepc`, cause code to `mcause`, privilege to `mstatus.MPP`, then jump to `mtvec`. Control must suppress `RegWrite` and `MemWrite` for the faulting instruction and flush the next fetch. In single-cycle this is just a mux forcing next PC to `mtvec` and gating writes with `trap`= 0. In pipelined machines it becomes a flush and bubble, covered in Ch. 4. Forgetting to gate `RegWrite` on trap lets a faulting `LW` with `rd`=`rs1` corrupt the base register even though the load never completed.

3.11 Don't-Care Optimisation and Truth-Table Minimisation

Rows marked "—" are don't-cares the synthesiser exploits. `MemToReg` for `SW` is irrelevant because `RegWrite`= 0, so `MemToReg` never reaches a register; the minimiser ties it to 0 to save a gate. `ALUSrc` for jumps is don't-care because the ALU result is discarded. Branch uses `ALUOp`= 01 which forces ALUCtrl to SUB regardless of funct, so funct bits are don't-cares for `BEQ`. Karnaugh maps for `RegWrite` show it is 1 only for R, LW, JAL/JALR: `RegWrite`=(R∨LW∨JAL), two OR gates. This is why clean don't-cares shrink area and power.

3.12 From Single-Cycle to Pipelined Control

Single-cycle control is combinational; pipelined control must ride along with the instruction. Each control signal is generated in ID but used in EX/MEM/WB, so it is latched into pipeline registers (ID/EX, EX/MEM, MEM/WB). The same truth table drives the pipeline, just delayed. This reveals why control hazards exist: a branch decision needs the Zero flag from EX, but the next fetch has already entered IF. Solutions are stalling, forwarding the flag, or predicting. Understanding the single-cycle truth table first makes these pipeline fixes obvious rather than mysterious.

3.13 Design Methodology

Steps: (1) list datapath points needing control, (2) decide instruction → signal mapping, (3) build truth tables, (4) minimise with Karnaugh or synthesis tool, (5) verify with exhaustive simulation. A single bug (e.g., enabling `RegWrite` during `SW` when `rd` overlaps `rs2`) silently corrupts state — formal verification of control is standard practice.

3.14 Worked Control Words: From Assembly to Signals

Derive the word for each of the five base opcodes, showing how don't-cares are chosen for minimal logic.

`ADD x5,x6,x7` (R, `0110011`): `RegWrite`=1, `ALUSrc`=0, `MemRead`=0, `MemWrite`=0. `MemToReg`=0, `Branch`=0, `Jump`=0, `ALUOp`=10, `ALUCtrl`=0010 (ADD). `ImmSrc` don't-care; tie to 00.

`LW x5,8(x6)` (I-load, `0000011`): `ALUSrc`=1 (imm), `MemRead`=1, `RegWrite`=1, `MemToReg`=1 (memory to register), `MemWrite`=0, `Branch`=`Jump`=0, `ALUOp`=00 (forces ADD for address), `ImmSrc`=00.

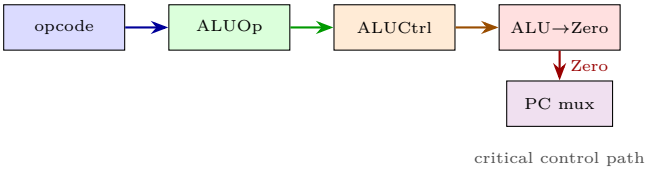
`SW x5,8(x6)` (S, `0100011`): `ALUSrc`=1, `MemWrite`=1, `MemRead`=0, `RegWrite`=0 (critical: if 1, `rd` field overlaps `rs2` encoding and would corrupt a register), `Branch`=`Jump`=0, `ALUOp`=00, `MemToReg`=−.

`BEQ x5,x6,off` (B, `1100011`): `ALUSrc`=0, `Branch`=1, `MemRead`=`MemWrite`=`RegWrite`=0, `ALUOp`=01 (forces SUB), `ImmSrc`=10 (B-type, imm[0]=0). `MemToReg`=−, `Jump`=0. Zero flag from ALU decides PC mux late.

`JAL x1,off` (J, `1101111`): `Jump`=1, `RegWrite`=1 (writes PC+4 to `rd`), `MemRead`=`MemWrite`=`Branch`=0, `ALUSrc`=−, `MemToReg`=− (actually selects PC+4 path, often encoded as 10 in 2-bit `MemToReg`), `ImmSrc`=11.

3.15 Control Timing and Why Branch Is Late

All control is combinational, so signals settle after opcode→PLA→mux. The PC mux is the last to settle because it waits for `Branch`∧`Zero`. Zero itself comes from the 32-bit ALU comparator ($R == 0$), a wide AND of NORs, so the path opcode→`ALUOp`→`ALUCtrl`→ALU→Zero→PC mux is one of the longest nets. If it violates setup at the PC register, branches jump to random addresses while loads still work, a confusing lab symptom.



3.16 Expanding the Truth Table: ADDI, LUI, AUIPC

The five-row table in the overview omits three common opcodes. `ADDI` (`0010011`, I-type) is like LW without memory: `ALUSrc`=1, `RegWrite`=1, `ALUOp`=00 (ADD), `MemRead`=0, `MemToReg`=0. `LUI` (`0110111`, U-type) writes `imm[31:12]`≪12 directly to `rd`: `RegWrite`=1, `ALUSrc`=−, `ALUOp`=11 (pass-through), `ImmSrc`=01. `AUIPC` (`0010111`) adds PC+U-imm: ALU does ADD with PC as one input, a separate PC→ALU mux controlled by `ALUSrcB`. The pattern is that every instruction is a row; synthesis merges rows with shared outputs.

3.17 Exam Checklist: Control

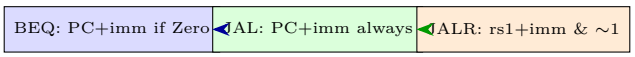
(1) list every signal and its width/meaning; (2) draw ALUOp→ALUCtrl decode table (00→ADD, 01→SUB, 10+funct→R-type); (3) write the main truth table for R/LW/SW/BEQ/JAL and mark don't-cares, explaining why each − is safe; (4) state hardwired = gates/PLA (fast) vs microcoded = ROM+μPC (flexible, multi-cycle); (5) describe trap: save `mepc`/`mcause`, jump `mtvec`, gate `RegWrite`/`MemWrite`, flush fetch.

3.18 Control Optimisation: Two-Level Decode

Why two levels (Main→ALUOp→ALUCtrl) instead of one flat table from opcode+funct to ALUCtrl? Because R-type is the only format that needs funct, and `ALUOp` compresses the choice: 00 means "always ADD" (loads/stores/ADDI), 01 means "always SUB" (branches), 10 means "look at funct". The second level is then only 4 rows. A flat table would be $7 \times 8 = 56$ product terms; the two-level version is $7 + 8 = 15$, smaller and faster. The synthesiser would discover the same factorisation, but writing it hierarchically makes the Verilog readable and the timing easier to close.

3.19 Jump Control: JAL vs JALR vs Branch

Branches are conditional and PC-relative; `JAL` is unconditional PC-relative; `JALR` is unconditional register-indirect. Control distinguishes them: `Branch`=1 means "use Zero", `Jump`=1 means "always take", `JumpReg`=1 means "target is ALU output not PC+imm". In single-cycle these three bits drive one 3-to-1 PC mux; in pipelined they become separate pipeline stages. Mixing up `JAL` and `JALR` targets is a frequent ISA exam trap: `JAL` offset is ± 1 MB (J-imm, imm[0]=0), `JALR` offset is ± 2 KB (I-imm) added to `rs1`.



three PC sources, one mux

3.20 Control Verification: Exhaustive Simulation

Control is small enough to verify exhaustively. Enumerate all 2^{10} combinations of opcode[6:0]+funct3, simulate the truth table, and compare to a golden reference from Spike. A Python checker does this in milliseconds. The most valuable test is the don't-care check: for any input where the spec says "−", assert that neither `RegWrite` nor `MemWrite` is spuriously asserted for opcodes that should not write. One stray `RegWrite`=1 on `SW` writes `rd`=imm[11:7] with memory data and silently corrupts the register file, a bug that passes `ADD` tests but fails `memcpy`.

3.21 X vs Don't-Care and Power

Don't-care is not "any value is fine in hardware" but "the synthesiser may pick whichever value minimises gates". Tying `MemToReg` to 0 for branches saves a mux input; tying `ALUSrc` to 0 for jumps saves a wire. Power also benefits: gating the ALU clock when `ALUOp`=11 (LUI pass-through) avoids toggling the adder. These micro-optimisations matter in a 32-bit MCU where the datapath is a large fraction of area.

3.22 Privileged Control and Trap Preview

User code is not the whole picture. The privileged ISA adds CSRs (`mstatus`, `mtvec`, `mepc`, `mcause`) and instructions `ECALL`, `EBREAK`, `MRET`, `CSRRW`. Control must now decode these opcodes, route CSR read/write data through the WB mux, and handle traps atomically as described above. A trap is a control-flow event that overrides the normal PC mux and write enables in the same cycle it is detected; failing to suppress `MemWrite` on a misaligned store that traps would corrupt memory at the faulting address. Pipelined control adds a trap pipeline register and a flush signal that turns later stages into NOPs.

3.23 ALU Decoder Truth in Full

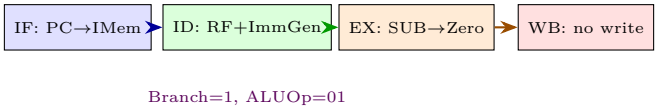
The textbook 8-row ALUCtrl table compresses to: if `ALUOp`=00 output 0010 (ADD); if 01 output 0110 (SUB); if 10 decode funct3/funct7: 000+0→0010 ADD, 000+1→0110 SUB, 111→0000 AND, 110→0001 OR, 100→0011 XOR, 010→0111 SLT, 011→1000 SLTU, shifts map to 1001–1011. Don't-care for non-R opcodes; the decoder ignores funct there so a load with stray funct bits still adds correctly. Verilog is a single `case` on {`ALUOp`,funct3,funct7[5]}.

3.24 Control and the Assembler: Pseudo-Instructions

`NOP` is `ADDI x0,x0,0`: control word is like ADDI but `RegWrite` is gated by `rd`≠0 so no write occurs, and the ALU does $0 + 0$. `MV rd,rs` is `ADDI rd,rs,0` with same word. `LI rd,imm` expands to `LUI+ADDI` when imm needs 32 bits, so control sees two real instructions. `CALL offset` is `AUIPC+JALR`. The control unit never sees pseudos; the assembler has already lowered them, so no extra rows are needed.

3.25 Putting It All Together: One Cycle, End to End

Follow `BEQ x5,x6,loop` at PC 0x1000 with offset −16 (loop at 0x0FF0). IF fetches `0xFE628AE3` (encoded B-imm −16); ID reads `x5`,`x6` and ImmGen makes `0xFFFFFFF0`; EX does SUB and Zero is 1 if equal; branch-adder made 0x0FF0 early; MEM does nothing; WB does nothing; at the next rising edge PC latches 0x0FF0 if Zero else 0x1004. Control asserted `Branch`=1, `ALUOp`=01, `RegWrite`=0, `MemRead`=0 the whole cycle. Every signal's purpose is visible in this one instruction.



3.26 Control Hazards Preview: Why Branches Hurt Pipelines

In single-cycle, the branch target and Zero are ready before the next PC latch, so no hazard exists. In a 5-stage pipeline, IF fetches the next instruction before ID even decodes the branch, and EX knows Zero two cycles later.

Without forwarding or prediction, the pipeline must stall two cycles per branch, losing 15–20% throughput. Solutions (branch prediction, early Zero in ID, delay slots) are all patches on the control timing we just derived. Mastering the single-cycle control word makes the pipeline fix obvious: move the comparison earlier or speculate and flush on mispredict.

3.27 Area and Power of Control

Control is tiny: the main PLA is ≈7 rows×8 columns, a few dozen gates. The ALU decoder is similar. Total control area is under 5% of the core; the datapath (memories, RF, ALU) dominates area and power. This is why RISC is efficient: simple truth tables need negligible energy compared to the SRAM arrays they steer. Adding microcode ROM would actually grow area for RV32I and slow it, so no one does it except for the rare complex CSR atomics.

3.28 Forward-Look: What Changes When We Pipeline

Single-cycle control is combinational; pipelined control latches ID's signals into ID/EX, EX/MEM, MEM/WB so they emerge alongside the data they steer. Branch's `Branch` and Zero meet at EX/MEM, not ID. Same truth table, only timing shifts, so Ch. 4 reuses every row.

The control unit is under 200 gates in a minimal RV32I; the datapath SRAM dominates area by 10×. Optimising control is about correctness, not area.

3.29 Bonus Drill: Control Word Table

For `ADDI x1,x0,10`; `ADD x2,x1,x1`; `SW x2,0(x0)`; `LW x3,0(x0)`; `BEQ x2,x3,done` the words (RegW, ALUSrc, MemR, MemW, M2R, Br, J, ALUOp) are: `ADDI`(1,1,0,0,0,0,0,00), `ADD`(1,0,0,0,0,0,0,10), `SW`(0,1,0,1,−,0,0,00), `LW`(1,1,1,0,1,0,0,00), `BEQ`(0,0,0,0,−,1,0,01). `SW` needs `RegWrite`=0 (else `rd`=imm[11:7] corrupts); `BEQ` needs `ALUOp`=01 for SUB.

3.30 Synthesis: How Small Control Really Is

Main control synthesises to a 7-row PLA of about 40 product terms; the ALU decoder is 8 rows. Together they are under 150 gates, versus >10k for the SRAM arrays. This is why RISC control is often drawn as a tiny box: it steers 10× its area in datapath. Verification is exhaustive: enumerate all 2^{10} opcode+funct combos against a Spike golden model.

Control's correctness is proved by exhaustive enumeration, not random testing, because the input space is only 10 bits.

3.31 Glossary and Signals You Must Know

Quick reference: Main Control (opcode → word), ALU Control (ALUOp+funct → ALUCtrl), `RegWrite`, `MemRead`, `MemWrite`, `MemToReg`, `ALUSrc`, `Branch`, `Jump` (see inventory), `ImmSrc` (format). Don't-care "−" lets synthesiser minimise; PLA is hardwired logic; μPC/μIR are micro sequencer; `mepc`/`mcause`/`mtvec` are trap CSRs.

3.32 Exercises

Exercise 3.1. Write the control-word for `ADDI x5,x6,42` and for `SW x5,8(x6)`. Which signals differ and why?

Exercise 3.2. Show that `RegWrite` for RV32I is 1 only for R, I-load, I-arith, JAL, JALR, LUI, AUIPC. Write its sum-of-products and minimise with don't-cares.

Exercise 3.3. Derive `ALUCtrl` for `SUB` vs. `ADD`: which funct bit distinguishes them and how does the decoder use it?

Exercise 3.4. Why is `MemToReg` a don't-care for branches and jumps? What would happen if it were set to 1 during `BEQ`?

Exercise 3.5. Compare hardwired and microcoded control for a new instruction that needs 4 micro-steps. Which is faster? Which is easier to patch post-silicon?

Exercise 3.6. Explain why control must suppress `RegWrite` when an exception is detected in MEM. What goes wrong if it does not?

Chapter 4

Pipelining & Hazards

Like a laundry pipeline: while one load dries, the next washes. Latency for one load is unchanged; throughput quadruples.

4.1 Latency versus Throughput: Why Pipeline?

A single-cycle datapath finishes one instruction per long clock: $T_{\text{seq}} = t_{\text{IF}} + t_{\text{ID}} + t_{\text{EX}} + t_{\text{MEM}} + t_{\text{WB}}$. The ALU idles during IF; the memory idles during EX. Pipelining overlaps: each stage works on a *different* instruction simultaneously, like an assembly line.

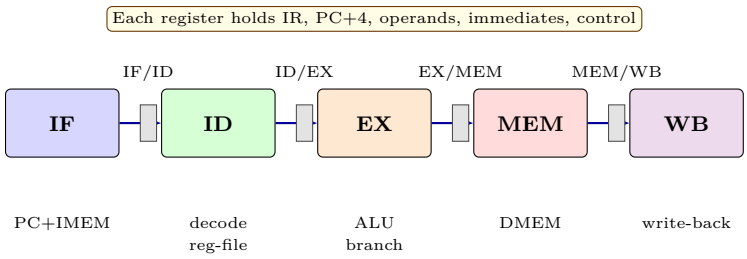
Two metrics separate cleanly. **Latency** is time for *one* instruction through all stages (still 5 cycles). **Throughput** is instructions completed per cycle in steady state (ideally 1/cycle). We pipeline to improve throughput at modest latency cost, not to make a single instruction faster. The laundry analogy is exact: one load takes 2 hours regardless, but with 4 stages you finish a load every 30 minutes in steady state.

Definition 4.1 (Ideal Speedup and Real Clock)**.** With $n = 5$ stages each t, k instructions: sequential time $= k \cdot n \cdot t$, pipelined time $= (n + k - 1) \cdot t$ (fill $n - 1$ cycles, then one per cycle). Speedup $= kn/(n + k - 1) \to n$ as $k \to \infty$. With unbalanced stage delays, clock period $= \max_i t_i + t_{\text{reg}}$, so real speedup $< n$. Register overhead t_{reg} covers setup, clk-to-Q, and skew.

Example: $t_{\text{IF}} = 2.0$, $t_{\text{ID}} = 1.5$, $t_{\text{EX}} = 2.0$, $t_{\text{MEM}} = 2.5$, $t_{\text{WB}} = 1.0\,\text{ns}$, register overhead $0.2\,\text{ns}$. Single-cycle $T = 9.0\,\text{ns}$. Pipelined $T_{\text{clk}} = \max t_i + 0.2 = 2.7\,\text{ns}$. Five instructions: sequential $45\,\text{ns}$, pipelined $(5 + 4) \times 2.7 = 24.3\,\text{ns}$, speedup $1.85\times$ not $5\times$ because the slowest stage (MEM at $2.5\,\text{ns}$) dictates. For $k = 1000$: sequential $9000\,\text{ns}$, pipelined $1004 \times 2.7 = 2711\,\text{ns}$, speedup $3.32\times$—approaching n but capped by imbalance.

4.2 The 5-Stage Pipeline and Its Registers

Stages: **IF** fetch (PC→IMEM), **ID** decode + reg-read, **EX** ALU/branch-compare, **MEM** D-memory, **WB** write-back. Between each pair sits a pipeline register that snapshots all state needed downstream: instruction word, PC+4, register operands, immediates, and control bits.



What each register carries: `IF/ID`: IR (32 b), PC+4, PC. `ID/EX`: Rs1/Rs2 values, imm, Rd, funct3/7, control (ALUSrc, ALUOp, MemRead/Write, RegWrite, Branch). `EX/MEM`: ALU result, Rs2 (store data), Rd, Zero/branch, control. `MEM/WB`: memory data, ALU result, Rd, RegWrite/MemToReg. Width ≈ 120–160 b total per register. On reset or flush, control bits are zeroed to create a `nop`.

Pipeline timeline (columns are cycles, rows are instructions in program order). Steady state is cycles 4–6 where all five stages are occupied.

Instr	1	2	3	4	5	6	7	8	9
`LW x5,0(x6)`	IF	ID	EX	MEM	WB				
`ADD x7,x5,x8`		IF	ID	EX	MEM	WB			
`SW x7,4(x9)`			IF	ID	EX	MEM	WB		
`SUB x10,x7,x1`				IF	ID	EX	MEM	WB	
`BEQ x10,x0,L`					IF	ID	EX	MEM	WB

Without hazards, CPI = 1 in steady state. Each column has exactly one instruction per stage; the diagonal shows one instruction flowing through the pipeline every cycle.

4.3 Three Hazard Classes

Pipelining would be trivial without dependencies. Real code creates hazards that force stalls or forwarding. Recall *latency* is unchanged (5 cycles); hazards hurt *throughput* by inserting dead cycles.

1. Structural hazards: two instructions need the same hardware simultaneously. Classic example: single-ported memory for IF and MEM—a load in MEM and the next fetch in IF collide. Fix: split I-MEM/D-MEM (Harvard at L1—separate I-cache and D-cache), or stall one stage. Modern cores avoid this by construction; the only remaining structural hazards are deliberate (e.g., one multiplier shared between EX stages in superscalar).

2. Data hazards (RAW): instruction j reads a register that instruction i has not yet written. Example: `ADD x5,x6,x7` → `SUB x8,x5,x9`—`SUB` needs `x5` in ID while `ADD` is still in EX. WAR/WAW hazards do not occur in this simple in-order 5-stage but appear in out-of-order pipelines.

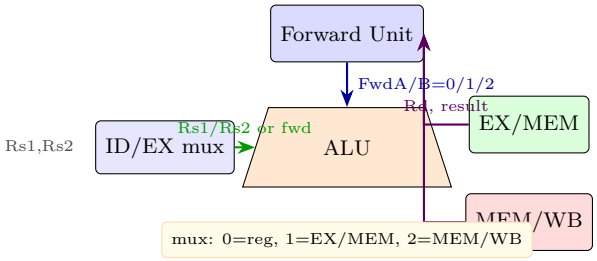
3. Control hazards: branch/jump target unknown until after EX (or ID if comparator moved). Instructions fetched after the branch may be wrong-path and must be flushed. Control hazards are rarer than data but cost more cycles per occurrence.

4.4 Forwarding: The Rd == Rs Truth Table

Rather than stall on every RAW, we *forward* (bypass) the result from a later stage directly to the ALU input mux. The hazard/forwarding unit compares destination and source register numbers combinatorially in ID.

Condition	Meaning	Forward?	Mux select
$\texttt{Rd}_{\texttt{EX}}$==$\texttt{Rs1}_{\texttt{ID}}$ and $\texttt{RegWrite}_{\texttt{EX}}$ and $\texttt{Rd}_{\texttt{EX}} \neq$x0	EX produces Rs1 needed now	Yes	FwdA=EX/MEM
$\texttt{Rd}_{\texttt{EX}}$==$\texttt{Rs2}_{\texttt{ID}}$ and $\texttt{RegWrite}_{\texttt{EX}}$ and $\texttt{Rd}_{\texttt{EX}} \neq$x0	EX produces Rs2 needed now	Yes	FwdB=EX/MEM
$\texttt{Rd}_{\texttt{MEM}}$==$\texttt{Rs1}_{\texttt{ID}}$ and $\texttt{RegWrite}_{\texttt{MEM}}$ and $\texttt{Rd}_{\texttt{MEM}} \neq$x0	MEM/WB produces Rs1	Yes*	FwdA=MEM/WB
$\texttt{Rd}_{\texttt{MEM}}$==$\texttt{Rs2}_{\texttt{ID}}$ and $\texttt{RegWrite}_{\texttt{MEM}}$ and $\texttt{Rd}_{\texttt{MEM}} \neq$x0	MEM/WB produces Rs2	Yes*	FwdB=MEM/WB
Otherwise	No match or x0	No	Fwd=Reg

*EX/MEM has priority over MEM/WB when both match (more recent result wins). The check Rd≠x0 is essential because x0 is hard-wired zero; forwarding a bogus value to x0 would corrupt it. Sources: EX/MEM forwards the ALU result; MEM/WB forwards either the ALU result or loaded data depending on MemToReg.



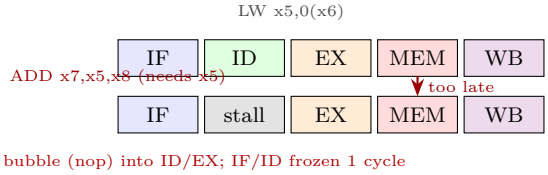
Code example with two forwardable hazards and one load-use that still stalls:

```
ADD x5,x6,x7        # EX/MEM -> next instr (forwarded)
SUB x8,x5,x9        # uses x5, forwarded from ADD, no stall
LW  x10,0(x5)       # load result only ready after MEM
ADD x11,x10,x1      # load-use: cannot forward in time -> 1 bubble
```

Without forwarding, the first two instructions would each stall 2 cycles; with forwarding they stall zero.

4.5 Load-Use Hazard: The Unavoidable Bubble

A load produces data at the *end* of MEM, but the next instruction needs it at the *start* of EX—one cycle too early. No forwarding path can go backward in time, so a one-cycle **bubble** (nop) is mandatory.



Detection logic evaluated in ID: if $\texttt{MemRead}_{\texttt{ID/EX}}$ is set and $\texttt{Rd}_{\texttt{ID/EX}}$ equals either $\texttt{Rs1}_{\texttt{IF/ID}}$ or $\texttt{Rs2}_{\texttt{IF/ID}}$, stall IF and ID for one cycle and inject nop (all-zero control) into ID/EX. After the bubble, forwarding from MEM/WB

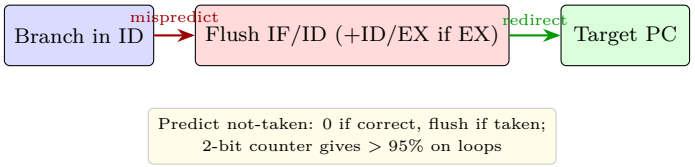
supplies the loaded word. Cost: one dead cycle per load-use pair. Compiler scheduling (hoist an independent instruction between `LW` and its use) can hide it entirely: `LW x5,0(x6)`; `ADD x7,x8,x9`; `ADD x10,x5,x7`.

4.6 Control Hazards and Branch Prediction: CPI = 1 + s

Branches decide late. In the baseline, comparison happens in EX, so the two instructions fetched after the branch (in IF and ID) are speculative. If the branch is taken, they must be flushed (convert to nops by zeroing IF/ID and ID/EX control).

Moving the comparator to ID (extra equality comparator plus early branch-target adder driven by ID's immediate) cuts penalty from 2 to 1 cycle: flush only IF/ID. This needs the register values to be ready in ID—a data hazard on a branch that immediately follows an ALU op may itself require a stall or forward to ID.

Prediction hides even that last cycle:



Performance under hazards is captured by one parameter s (average stall cycles per instruction):

$$\text{CPI} = \text{CPI}_{\text{ideal}} + s = 1 + s, \qquad s = s_{\text{data}} + s_{\text{control}} + s_{\text{structural}}.$$

For branches: let f_b be branch frequency, m mispredict rate, penalty c per mispredict. Then $s_{\text{control}} = f_b \cdot m \cdot c$. Example with baseline EX-resolved branches: $f_b = 20\%$, $m = 30\%$ (predict-not-taken), $c = 2$ gives $s = 0.20 \times 0.30 \times 2 = 0.12$, so CPI = 1.12. With a 2-bit predictor and ID-resolved branches: $m = 5\%$, $c = 1$ gives $s = 0.20 \times 0.05 \times 1 = 0.01$, so CPI = 1.01—almost ideal. Modern predictors (2-bit counters, correlating/tournament, TAGE) dominate branch-heavy performance; accuracy, not pipeline depth, sets CPI.

Forwarding minimises s_{data}; branch prediction minimises s_{control}; split I/D caches minimise $s_{\text{structural}}$. Together they keep $s \ll 1$.

4.7 Worked Hazard Walkthrough

Trace `LW x5,0(x1)`; `ADD x6,x5,x2`; `SUB x7,x6,x3`; `BEQ x7,x0,Loop` through the pipeline with full forwarding and ID-resolved branches.

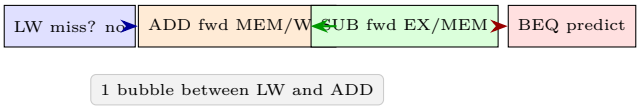
Cycle 1–2: `LW` in IF, then ID. No hazard yet.

Cycle 3: `LW` in EX, `ADD` in ID. Hazard unit sees $\texttt{MemRead}_{\texttt{ID/EX}}$ and `Rd`=`x5` equals $\texttt{Rs1}_{\texttt{IF/ID}}$—load-use detected. Action: freeze PC and IF/ID, inject bubble into ID/EX. `ADD` is held in ID one extra cycle.

Cycle 4: `LW` in MEM, bubble in EX, `ADD` still in ID (now safe to forward from MEM/WB next cycle). No new hazard.

Cycle 5: `LW` in WB (writes x5), `ADD` in EX (receives x5 forwarded from MEM/WB), `SUB` in ID. `SUB` needs `x6` which `ADD` will produce in EX/MEM next cycle—forwardable, no stall. Comparator in ID for the later `BEQ` will need `x7`, creating a similar forward chain.

Result: one bubble for the load-use, zero for the ALU-ALU chain, one flush if the branch is mispredicted. CPI $= 1 + 1/4 = 1.25$ for this snippet without branch mispredict; with correct prediction CPI $= 1.25$, with mispredict $= 1.50$.



4.8 Stall Minimisation Toolbox

Compiler and hardware cooperate to keep s small: (1) **schedule** independent instructions between load and use (often found by unrolling loops), (2) **unroll** to expose more independent work, (3) **predict** branches with 2-bit or TAGE predictors, (4) **speculate** past branches and squash on miss, (5) **superscalar** to find parallelism across wider issue. Each technique is judged by ΔCPI versus ΔT_{clk}—if it raises clock period more than it cuts CPI, it loses.

4.9 Why Forwarding Sometimes Fails and How Compilers Help

Forwarding covers ALU→ALU and MEM→ALU paths, but two cases still stall. First, load-use as above: data arrives from DRAM too late. Second, branch data hazard: `ADD x5,x6,x7`; `BEQ x5,x8,Label` needs `x5` in ID for comparison, but `x5` is still in EX—requires a forward to ID or a one-cycle stall if the bypass to ID is not implemented. Compilers hide load-use by scheduling: place an independent instruction between the load and its consumer, e.g. unroll a loop to interleave iterations. When no independent work exists, the bubble is unavoidable and accounted for in s_{data}.

4.10 Branch Predictor Detail: 2-bit Saturating Counter

Each branch PC indexes a table of 2-bit counters: states 00 strongly not-taken, 01 weakly not-taken, 10 weakly taken, 11 strongly taken. Taken increments toward 11, not-taken decrements toward 00; prediction is taken if state ≥ 10. This filters single outliers: a loop branch taken 9 times then not-taken once stays predicted taken, unlike a 1-bit predictor that would mispredict twice. With 20% branches and per-branch mispredict 5%, $s_{\text{control}} = 0.20 \times 0.05 \times 1 = 0.01$ (ID-resolved), versus 0.12 without prediction—an order-of-magnitude CPI win.

4.11 Advanced Glimpse: Superscalar and Out-of-Order

Real cores fetch/decode 4–8 instructions per cycle (superscalar), rename registers to remove WAR/WAW false dependencies, and execute out-of-order via Tomasulo reservation stations, retiring in-order for precise excep-

tions. Deeper pipelines (14–19 stages) raise f but amplify hazard penalties, so the same trade-off T_{clk} vs. CPI governs. The hazard taxonomy above scales: forwarding becomes a full bypass network with many read ports, branch prediction becomes TAGE with indirect targets and return stacks.

4.12 Exercises

Exercise 4.1. For `LW x5,0(x1)`; `ADD x6,x5,x2`; `SUB x7,x6,x3`, list RAW hazards. Which are resolved by forwarding, which need a bubble? Show the timeline with and without forwarding.

Exercise 4.2. Why does load-use need a bubble even with full forwarding? Draw the cycle diagram and mark when data is available vs. needed.

Exercise 4.3. A pipeline has branch penalty 2 cycles, 20% branches, 70% predicted correctly. Compute s_{control} and CPI. If moving compare to ID cuts penalty to 1, what is new CPI?

Exercise 4.4. Explain why `Rd==x0` must suppress forwarding. Give a code snippet where omitting the check corrupts `x0`.

Exercise 4.5. A 5-stage pipeline with $t_{\text{IF}} = 2$, $t_{\text{ID}} = 1.5$, $t_{\text{EX}} = 2$, $t_{\text{MEM}} = 2.5$, $t_{\text{WB}} = 1\,\text{ns}$ plus $0.2\,\text{ns}$ register overhead. What is T_{clk}? Latency for one instruction? Throughput for 1000 instructions? Speedup vs. single-cycle?

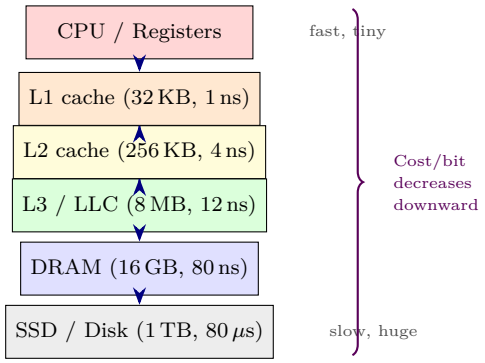
Chapter 5

Memory Hierarchy & Caches

Bridging the processor–memory gap: keep what you use close, and you rarely wait.

5.1 The Memory Hierarchy

Processor speed doubles every ≈ 2 years; DRAM latency improves $\approx 7\%$ per year. Without intervention, the CPU idles waiting for memory. The hierarchy exploits **locality**: temporal (reuse soon) and spatial (neighbours soon) to keep hot data in small, fast memories. This is why a 3 GHz core can sustain several instructions per cycle even though DRAM is ~ 80 ns away: over 95% of accesses hit in L1/L2.



Inclusive caches duplicate; exclusive do not. Modern chips have split L1 I/D caches to avoid structural hazards. **Temporal locality** examples: loop variables, stack data, recently called functions. **Spatial**: sequential instruction fetch, array traversal, next struct field. A 64 B block captures the next 15 words spatially for free once one word misses.

5.2 Cache Fundamentals

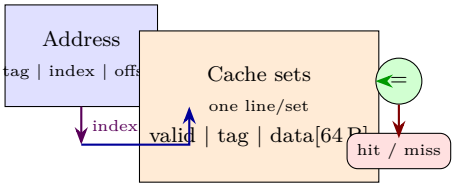
A cache holds **blocks** (lines) of B bytes (typically 64 B), each with a **tag**, **valid** bit, and optionally **dirty** bit. On access, address splits into `[tag | index | offset]`. Index selects a set; tag is compared; offset selects byte within block. Hit: data returned in cache latency. Miss: block fetched from next level, possibly evicting another.

Miss types (3 Cs): **compulsory** (first touch), **capacity** (cache too small), **conflict** (associativity too low). Miss rate m, hit time h, miss penalty p: AMAT $= h + m \cdot p$ (average memory access time).

Choosing B: larger B exploits spatial locality and amortises tag overhead but increases miss penalty (more bytes moved) and false sharing. 32–128 B is the sweet spot; 64 B dominates today because it matches DRAM burst length (8×8 B).

5.3 Direct-Mapped Cache

One block per set; index = (block address) mod (#sets). Simplest, but conflict-heavy.

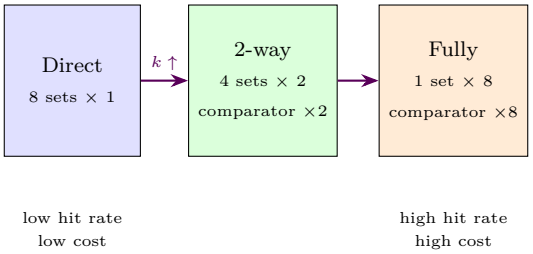


Example: 32 KB direct, 64 B blocks ⇒ 512 sets, 9 index bits, 6 offset bits, tag $= 32 - 9 - 6 = 17$ bits.

5.4 Set-Associative & Fully Associative

k-way set-associative: index selects a set containing k lines; all k tags compared in parallel; any block maps to one set but any line within it. $k = 4$ or 8 is typical; higher k reduces conflicts with diminishing returns and higher power.

Fully associative: one set with all lines — no index, any block anywhere; needs parallel tag compare across all lines (content-addressable). Rare except for small TLBs or victim caches.



Replacement: LRU (exact or pseudo-LRU tree), FIFO, random. LRU wins for loops but needs age bits.

Write policies: write-through (to next level immediately; simple but bandwidth-heavy) vs. write-back (dirty bit, write only on eviction; needs coherence handling). Write-allocate (fetch block on write miss) vs. no-write-allocate.

5.5 Address Decomposition: Worked Example `0x12345678`

Take 32 KB direct-mapped, 64 B blocks, 32-bit byte addresses. This is the canonical NTU exam configuration.

First compute geometry. Number of blocks: 32 KB/64 B = 512 lines. Direct-mapped so #sets = 512. Block offset bits: $\log_2 64 = 6$ bits (byte within block). Index bits: $\log_2 512 = 9$ bits. Tag bits: $32 - 9 - 6 = 17$ bits. Each line stores 17-bit tag + 1 valid + 1 dirty + 64 B data ≈ 67 B.

Now decompose `0x12345678`. In binary: `0001 0010 0011 0100 0101 0110 0111 1000`.

Tag (17 b)	Index (9 b)	Offset (6 b)
`0x02468` = 0000 1001 0011 00010	`0x159` = 1 0101 1001	`0x38` = 11 1000

Check: hex `0x12345678` >> 6 =`0x0048D159` (block number); `0x0048D159` mod 512 = 512×`0x91A2` remainder `0x159` (345 decimal) ⇒ set 345. Tag = 512-block number >> 9 = `0x02468`. So this address hits iff set 345 is valid and its tag equals `0x02468`; otherwise the whole 64 B block `[0x12345640-0x1234567F]` is fetched. Quick sanity trick: offset = low hex digit masked to 6 bits. Since $64 = 2^6$, the low 6 bits are within-hex-digit arithmetic: `0x78` = `0b01111000`, so offset `0x38` is correct; 64 B alignment means `0x12345678` is 56 B into its block.

For a 4-way cache of same size (512/4 = 128 sets, 7 index bits, 8 tag bits wider): index = (`0x0048D159` mod 128) = `0x59` = 89, tag = `0x048D1`. One extra tag bit compensates for fewer index bits—trade conflict rate for comparator cost.

5.6 The Three Cs: Why Misses Happen

Compulsory (cold) misses are unavoidable on first reference to a block. For 64 B blocks scanning a 1 MB array, compulsory misses ≈ 1MB/64B = 16384 regardless of associativity. Prefetching can hide but not eliminate them.

Capacity misses occur because the working set exceeds cache size. Example: looping over a 64 KB array with a 32 KB cache—every block is evicted before reuse, so hit rate ≈ 0% even with fully associative placement. Doubling cache size would eliminate these misses; doubling associativity would not.

Conflict misses occur when k-way associativity is too low. Classic: direct-mapped 32 KB cache, stride 32 KB = 512 blocks. Addresses `0x00000000`, `0x00008000`, `0x00010000` all map to set 0. A loop touching two such addresses ping-pongs forever with 100% miss rate; 2-way associativity fixes it. Conflict misses are the only Cs that associativity reduces. Rule of thumb on increasing block size B: compulsory ↓ (fewer blocks to touch), capacity ↓ slightly, conflict ↑ (fewer sets). Too-large B pollutes cache with unused neighbours.

5.7 Replacement and Write Policies in Depth

LRU: evict least-recently-used line in the set. True LRU needs $\log_2(k!)$ bits per set—expensive for $k > 4$. **Pseudo-LRU** (tree-PLRU) uses $k - 1$ bits: a binary tree of preference bits flipped on access. Almost as good as LRU for loops, far cheaper. **FIFO** and **random** need no age tracking; random avoids pathological stride patterns and is common in L1.

Write-through: every store writes through to next level (write buffer hides latency). Simple, keeps lower levels coherent, but burns bandwidth: a tight store loop at 3 GHz can saturate the L2 port. **Write-back**: stores

set dirty bit only; eviction writes back the whole 64 B block. Saves 3–10× bandwidth for write-heavy code but needs coherence protocol (MESI) on multi-core.

Write-allocate pairs with write-back: on write miss, fetch the block, then overwrite the word. Avoids a future miss on the same block. **No-write-allocate** (often with write-through) just forwards the word and does not fetch—better when you will overwrite the entire block anyway (e.g., `memset`).

5.8 Cache Performance: AMAT Single- and Multi-Level

Single-level AMAT is intuitive: AMAT $= h + m \cdot p$. Let L1 $h = 1$ ns, $m = 5\%$, $p = 80$ ns to DRAM:

$$\text{AMAT} = 1 + 0.05 \times 80 = 5.0\,\text{ns}.$$

CPI impact: if 30% of instructions are loads/stores each needing one access, memory stall $\approx 0.3 \times (5 - 1) = 1.2$ cycles per instruction. Add L2: let $h_{L2} = 4$ ns, local miss rate $m_{L2} = 20\%$ (fraction of L1 misses that also miss L2), DRAM 80 ns. Hierarchical AMAT nests:

$$\text{AMAT} = h_{L1} + m_{L1} \cdot (h_{L2} + m_{L2} \cdot p_{\text{DRAM}}).$$

$$= 1 + 0.05 \times (4 + 0.20 \times 80) = 1 + 0.05 \times 20 = 2.0\,\text{ns}.$$

Hierarchy cut AMAT from 5 ns to 2 ns—adding a 4 ns L2 more than pays for itself because it filters 80% of DRAM trips. Global L2 miss rate $= m_{L1} \cdot m_{L2} = 1\%$.

Example with numbers the examiner loves: $h_{L1} = 2$ cycles, $h_{L2} = 10$ cycles, $m_{L1} = 4\%$, $m_{L2} = 25\%$, $p = 200$ cycles. AMAT $= 2 + 0.04 \times (10 + 0.25 \times 200) = 2 + 0.04 \times 60 = 4.4$ cycles. Without L2: $2 + 0.04 \times 200 = 10$ cycles.

Matrix traversal (row- vs. column-major) ties locality to AMAT. Consider 1024×1024 `int` (4 B), row-major in C, 64 B blocks ⇒ 16 ints/block. Row-major: inner loop strides by 4 B, so 1 miss per 16 accesses ⇒ 65536 misses. Column-major: stride $= 1024 \times 4\,\text{B} = 4096\,\text{B} = 64$ blocks ⇒ every access misses ⇒ 1048576 misses, 16× worse. That 16× feeds directly into m and AMAT, so column-major can be 3–8× slower in wall time despite identical instruction count.

5.9 Fully Associative vs. Direct: When Each Wins

Direct-mapped needs one comparator, minimal power, deterministic index — ideal for L1 where latency is critical and h dominates. Fully associative needs N parallel comparators and a CAM; search is slower and hotter, so it only appears for small structures (32–64-entry TLB, victim cache, or 8-entry store buffer). Set-associative $k = 4$–8 is the practical middle ground for L2/L3: conflict rate within 2% of fully associative, power only k comparators.

	Direct	4-way	Fully assoc. (512 entries)
Comparators	1	4	512
Hit latency	1.0 ns	1.2 ns	~ 3 ns
Miss rate (SPECint)	5.1%	3.4%	3.1%
Power / access	1×	2.5×	60×

5.10 Locality in Code: Why Caches Work

Consider summing an array: `for (i=0;i<N;i++) sum+=A[i]`. Temporal locality: `sum` and `i` reused every iteration, so they stay in registers/L1. Spatial locality: `A[i]` and `A[i+1]` share a 64 B block, so 15 of 16 accesses hit. Contrast a linked list traversal where nodes are scattered: spatial locality collapses and miss rate approaches 100%. Caches reward predictable, sequential access patterns—the common case.

Compulsory misses are the price of the first touch; hardware prefetchers hide them by fetching the next block while the core computes. Stride prefetchers detect constant strides (e.g., +64 B) and stay one block ahead, cutting compulsory misses for array loops by $> 80\%$.

5.11 Putting It Together: Row-Major Simulation

Simulate $A[4][8]$ of `int` (4 B), 32 B blocks (8 ints), direct-mapped 64 B cache (only for illustration, tiny). Row-major layout stores $A[0][0..7]$ contiguously, then $A[1][0..7]$, etc. Accessing row 0 touches two blocks: $A[0][0..7]$ (one miss, seven hits). Column-major would touch $A[0][0], A[1][0], A[2][0], A[3][0]$ which lie 32 B apart (8 ints × 4 B), each in a different block—four misses for four accesses. Scale to 1024×1024: row-major reuses each block 16 times, column-major reuses zero times until eviction. This is why loop interchange (swap loop order) is the single most effective cache optimisation.

5.12 Cache Optimisations and Victim Caches

Four classic tweaks reduce AMAT without growing the cache: (1) **early restart / critical-word-first**: forward the requested word to the CPU as soon as it arrives, fill the rest of the block in the background; (2) **write buffer**: coalesce write-through stores so the core does not stall; (3) **prefetching**: hardware or `__builtin_prefetch` hints fetch ahead; (4) **victim cache**: a tiny fully-associative buffer (4–8 entries) holds recently evicted blocks, catching conflict misses at ~ 1 ns.

Blocking (tiling) reorders nested loops to keep a $B \times B$ tile in cache. For matrix multiply $C = A \times B$, naive order streams B column-wise (poor locality); tiling with 32×32 tiles keeps all three tiles in L1, cutting misses 4–8× and often doubling performance. This optimisation turns a capacity/conflict problem into compulsory-only within the tile.

5.13 Why Hierarchy Beats Flat Memory

A flat DRAM-only memory forces every access to pay 80 ns. At 3 GHz that is 240 cycles—the core would retire one instruction then stall for 239 cycles. The hierarchy amortises this by filtering accesses through caches: if L1 hits 95% of the time at 1 ns and L2 catches 80% of the remaining misses at 4 ns, average cost drops from 80 ns to about 2 ns. The principle is simple: make the common case fast and tolerate the rare slow case. This is the same reason we keep a phone's recent contacts on the home screen and the full address book behind a search.

Temporal locality arises because programs reuse data quickly. Loop counters, stack frames, and recently called functions are accessed repeatedly within microseconds. Spatial locality arises because code and data are laid out sequentially: fetching instruction n makes $n + 1$ likely, and scanning an array touches neighbours. Hardware exploits both by moving whole 64 B blocks on a miss, not just the requested word. Even a single miss therefore prefetches 15 neighbouring words for free.

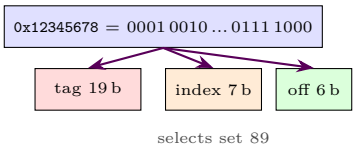
5.14 Miss-Rate Sensitivity and Block Sizing

Consider compulsory misses for a cold scan of S bytes with block size B: misses $= S/B$. Doubling B halves compulsory misses but also halves the number of distinct blocks the cache can hold (C/B), increasing conflict and capacity misses. There is a U-shaped curve: miss rate falls as B grows from 16 B to 64 B, then rises beyond 128 B as pollution dominates. Designers pick 64 B because it matches DRAM burst length (8 beats $\times$ 8 B) and balances the three Cs.

Capacity example revisited: array A of 8 KB fits in a 32 KB cache with room to spare, so after the first scan it hits entirely. Array B of 64 KB does not fit; a direct-mapped cache thrashes even if fully associative, because no placement can avoid eviction before reuse. The fix is not associativity but either a larger cache (L2/L3) or algorithmic tiling that partitions B into 16 KB tiles that do fit. This is why matrix libraries tile.

5.15 Address Decomposition Revisited: 4-Way Example

For the same 32 KB cache as 4-way set-associative, geometry changes. Total lines = 512, lines per set = 4, so sets = 128, index bits = $\log_2 128 = 7$, offset = 6, tag = 19 bits. Decompose `0x12345678` again: block number `0x0048D159`, set = `0x0048D159` mod 128 = `0x59` = 89 decimal, tag = `0x0048D159` >> 7 = `0x91A2` truncated to 19 bits = `0x091A2`. The cache compares four tags in set 89 in parallel; a hit needs any of the four to equal `0x091A2` and be valid. Wider tag costs one extra bit per index bit removed, but fewer sets means fewer conflict aliases.



5.16 Write Path Walkthrough

Trace two stores to the same block under write-back, write-allocate. Initially block is invalid. Store to `0x12345678` misses, fetches block `[0x12345640-7F]`, writes word at offset `0x38`, sets dirty= 1. Second store to `0x12345644` (same block, offset `0x04` within 64 B) hits, updates a different word, dirty stays 1. No DRAM traffic yet. When the block is later evicted to make room for `0x12347678` (same set, different tag), the 64 B dirty block is written back in one burst. Total DRAM writes: one 64 B burst instead of two 4 B writes.

Under write-through, no-write-allocate, the first store misses and writes 4 B directly to L2/DRAM without fetching the block. The second store hits in L2 but still writes through. Two 4 B DRAM writes occur, and the block never enters L1—subsequent reads to the same block will miss again. This is why write-through pairs with no-write-allocate for streaming writes like `memset`, while write-back pairs with write-allocate for general code.

5.17 AMAT Drill: From Paper to Code

Exam-style drill. Given: L1 $h = 1$ cycle, $m = 6\%$, L2 $h = 8$ cycles, local $m_2 = 30\%$, DRAM $p = 200$ cycles. Compute AMAT with and without L2. Without L2: $1 + 0.06 \times 200 = 13$ cycles. With L2: $1 + 0.06 \times (8 + 0.30 \times 200) = 1 + 0.06 \times 68 = 5.08$ cycles. Speedup $13/5.08 \approx 2.56\times$. If the core issues one memory access per 2 instructions at 3 GHz, memory stall per instruction drops from 6.5 to 2.54 cycles, saving about 1.3 ns per instruction or roughly 4 ns per load.

Sensitivity: halving m_1 from 6% to 3% saves $0.03 \times 68 = 2.04$ cycles, more than doubling L2 size might achieve. This is why compiler optimisations that improve locality (tiling, loop interchange) often beat hardware changes. Always attack the highest $m \cdot p$ product first.

5.18 Prefetching and Non-Blocking Caches

A blocking cache stalls the core on every miss until data returns. A non-blocking (lockup-free) cache allows hits under miss: the core continues executing independent instructions while the miss is serviced, tracking outstanding misses in MSHRs (miss status holding registers). With 4 MSHRs, four concurrent misses can be in flight, hiding much of DRAM latency for parallel workloads. Prefetching compounds this: a stream prefetcher that fetches $A[i + 1]$ when $A[i]$ misses converts compulsory misses into hits if the prefetch arrives before the demand access. Accuracy and coverage trade off: aggressive prefetching reduces misses but wastes bandwidth on mispredicted streams.

5.19 Cache Coherence Glimpse

On multi-core, each core has private L1/L2. If core 0 writes x cached by core 1, core 1 must see the new value. Coherence protocols (MSI/MESI) track per-block states: Modified (dirty, exclusive), Exclusive (clean, exclusive), Shared (clean, may be in other caches), Invalid. A write to Shared triggers an invalidate broadcast; readers then miss and fetch the new block. This is why write-back caches need extra messages on writes that hit

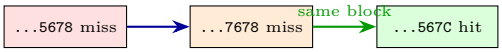
Shared—the cost of keeping private caches coherent. False sharing occurs when two cores write different words in the same 64 B block: each write invalidates the other's copy, ping-ponging the block at 80 ns per transfer and destroying performance. Padding structs to 64 B or partitioning data per core avoids it.

5.20 Exercise Walkthrough: AMAT Sensitivity

Worked example for exam technique. L1: $h = 2$ ns, $m = 8\%$. L2: $h = 6$ ns, local $m = 25\%$. DRAM: 80 ns. Compute AMAT with L2 nested: $2 + 0.08 \times (6 + 0.25 \times 80) = 2 + 0.08 \times 26 = 4.08$ ns. If you can either halve L1 miss rate to 4% (better locality) or halve L2 miss rate to 12.5% (larger L2), which is better? Halving m_1: $2 + 0.04 \times 26 = 3.04$ ns (saves 1.04 ns). Halving m_2: $2 + 0.08 \times (6 + 10) = 3.28$ ns (saves 0.80 ns). Improving L1 wins because its misses are more frequent. Always prioritise the level with larger $m \cdot p$ contribution.

5.21 Detailed Cache Trace: Three Addresses

Trace accesses to 32 KB direct cache (512 sets, 64 B blocks) in order: `0x12345678`, `0x12347678`, `0x1234567C`. First access `0x12345678`: set 345, tag `0x02468`, miss, fetch block `0x12345640-7F`, install tag. Second access `0x12347678`: block number `0x048D1D9`, set = `0x1D9` = 473, tag `0x02468` also but different set, so no conflict—miss, fetch `0x12347640-7F` into set 473. Third access `0x1234567C`: same block as first access (offset `0x3C` vs `0x38`), set 345, tag matches, hit in 1 ns. If instead second address were `0x123C5678` (set 345 again, tag `0x024F1`), it would evict the first block; the third access would then miss—a conflict miss that 2-way associativity would have avoided by keeping both tags in set 172 (rehashed for 4-way).



5.22 AMAT Drill: Two-Level Table

Tabulate AMAT for quick exam reference (all times in ns or cycles consistently): L1 hit 1, L2 hit 8, DRAM 80, L1 miss 5%, L2 local miss 20% → AMAT $= 1 + 0.05(8 + 0.20 \times 80) = 2.0$. If DRAM doubles to 160 (slower DIMM): AMAT $= 1 + 0.05(8 + 32) = 3.0$—still 2.7× better than no L2 ($1 + 0.05 \times 160 = 9.0$). If L1 miss climbs to 10% (pointer-chasing workload): AMAT $= 1 + 0.10(8 + 16) = 3.4$. Lesson: workload miss rate dominates technology latency; no cache hierarchy saves a workload with no locality.

5.23 Inclusive vs. Exclusive and Victim Effects

Inclusive caches duplicate every L1 block in L2/L3, so L3 size is shared with L1/L2 and effective capacity is just L3 size. Exclusive caches do not duplicate, so effective capacity is sum of levels, but a hit in L1 that misses in L2 must still check L2. Modern Intel L3 is non-inclusive: it behaves inclusively for hot data but can evict without invalidating L1, balancing capacity and simplicity. Victim caches change the picture: a miss in L1 that hits in the 8-entry victim cache costs ~ 2 ns instead of 80 ns, recovering many conflict misses cheaply. The AMAT with a victim cache of hit rate v among L1 misses becomes $h_1 + m_1 \cdot ((1 - v) \cdot p + v \cdot h_{\text{victim}})$.

The choice of inclusivity also affects coherence traffic. With inclusive L3, the L3 knows every block present in any L1, so snoop filters are simple: check L3, and if absent, no L1 need be probed. With exclusive or non-inclusive hierarchies, a directory or snoop broadcast is needed, costing interconnect bandwidth. This is why large multi-core chips often pay the capacity cost of inclusive or directory-based designs.

5.24 Quantitative Locality Tuning

Loop tiling example with numbers. Multiply 512×512 doubles (8 B), cache 32 KB can hold about 4096 doubles. Naive triple loop streams one matrix column-wise, missing every access after the first tile. Tiled with 64×64 tiles: each tile is $64 \times 64 \times 8\,\text{B} = 32\,\text{KB}$, exactly fitting. Each tile is loaded once per outer iteration, so misses drop from $O(n^3/B)$ to $O(n^3/(B\sqrt{C}))$, roughly 4× fewer for $n = 512$. Measured speedups of 1.8–3× are typical before any vectorisation, purely from locality.

Prefetch distance tuning: if memory latency is 80 ns and compute does 20 cycles (6.7 ns at 3 GHz) per iteration, the prefetcher must stay $80/6.7 \approx 12$ iterations ahead. Too close and data arrives late; too far and it pollutes or is evicted before use. Compilers use profile-guided prefetch distance for this reason.

5.25 Exam Checklist: Cache

For any cache question, follow this order and you will not lose marks: (1) compute B, #blocks, #sets, index bits $= \log_2(\#\text{sets})$, offset bits $= \log_2 B$, tag bits = addr bits − index − offset; (2) split the given hex address: block number = addr>> offset, index = block number mod #sets, tag = block number >> index bits; (3) classify miss: first touch → compulsory, working set > cache → capacity, else conflict (fixable by associativity); (4) apply AMAT: $h + m \cdot p$, nesting for multi-level, and remember m_2 in the formula is local to that level, global is $m_1 m_2$; (5) for row/column-major, count blocks touched: row-major $= N^2/(B/4)$ misses, column-major $= N^2$ misses for $B = 64$, 4 B ints. Common pitfall: mixing global and local m_2. Always label which you use. Another pitfall: forgetting that block offset is byte offset, so 64 B needs 6 bits even for word-addressed views.

Additional row-major drill: 1024×1024 int, 64 B blocks, row-major misses $= 1024 \times (1024/16) = 65536$, column-major $= 1024 \times 1024 = 1048576$, ratio 16×. Prefetch bonus: with next-block prefetch, row-major compulsory misses halve if prefetcher stays one block ahead. Associativity drill: direct $m = 5.1\%$ vs 4-way $m = 3.4\%$ at $h = 1$ ns, $p = 80$ ns gives AMAT 5.08 ns vs 3.72 ns, saving 1.36 ns per access. LRU vs random: for cyclic access $ABCABC\ldots$ over 3 blocks in a 2-way cache, LRU hits 0%, random hits ≈ 25%—random avoids LRU's pathological resonance. Write-buffer depth matters: 4-entry buffer hides 4 consecutive write-through misses; the fifth stalls until one drains. Victim cache of 8 entries catches 40–60% of conflict misses in direct-mapped L1, almost matching 2-way at lower power. Tiling 32×32 on 512×512 double keeps 8 KB tiles in L1, cutting capacity misses 4× and doubling DGEMM throughput. End-to-end: locality → block size → AMAT → CPI → wall time; every optimisation ultimately reduces $m \cdot p$.

Row-major vs column-major gap widens with larger N: for 2048×2048 the ratio stays 16× but absolute stall doubles. False sharing fix: pad shared counters to 64 B or partition per core; otherwise ping-pong costs 80 ns per write. Non-blocking L1 with 4 MSHRs hides up to 4 misses concurrently; MLP matters more than raw m for out-of-order cores. Critical-word-first cuts load-use penalty from 80 ns to ∼ 15 ns for the requested word,

even though block fill still takes 80 ns. Directory coherence replaces broadcast with point-to-point invalidates, scaling to 64+ cores where snooping would saturate.

5.26 Exercises

Exercise 5.1. A 16 KB direct cache, 32 B blocks, 32-bit addresses. How many sets? How many tag bits? If address 0x12345678 is cached, what are its index and tag in hex?

Exercise 5.2. Compare AMAT for direct ($m = 6\%$) vs. 4-way ($m = 3.5\%$) with $h = 1$ ns, $p = 70$ ns. Which wins and by how much? What extra hardware does 4-way need?

Exercise 5.3. Why does write-back need a dirty bit while write-through does not? Give a sequence where write-back saves bandwidth.

Exercise 5.4. Explain the 3 Cs with a concrete loop that exhibits each. How does doubling block size affect each C?

Exercise 5.5. For a 64 B block, why does row-major traversal of a 1024×1024 int array miss far less than column-major? Quantify misses per row/column.

Chapter 6

Virtual Memory

Every program thinks it owns all of memory; the OS and hardware conspire to make it so.

6.1 Why Virtual Memory?

Three problems without it: (1) programs must know physical layout and avoid each other, (2) memory looks discontinuous due to fragmentation, (3) isolation is weak — one bug corrupts another process. Virtual memory solves all three by giving each process its own virtual address space, translated by hardware to physical frames, with protection checked on every access.

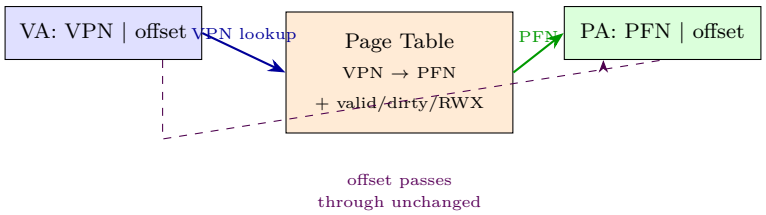
Benefits: **isolation** (page-table permissions), **sharing** (same physical frame mapped in two spaces), **demand paging** (pages loaded on first touch), and the illusion of more memory than DRAM via swapping to disk. Without VM, `malloc` would need contiguous physical frames; with VM, any scattered frames appear contiguous virtually. This is why a 4 KB `mmap` always succeeds even when DRAM is heavily fragmented.

6.2 Pages, Frames, and Page Tables

Memory is divided into fixed **pages** (virtual, e.g., 4 KB) and **frames** (physical, same size). A **page table** maps virtual page number (VPN) $\rightarrow$ physical frame number (PFN) plus metadata (valid, dirty, permission bits R/W/X, accessed, user/supervisor).

For 32-bit VA, 4 KB pages: offset = 12 bits, VPN = 20 bits. A single-level table needs 2^{20} entries (≈ 4 MB) — too large and sparse. So we use multi-level (hierarchical) tables: RISC-V Sv32 has two levels (10+10+12), Sv39 three levels (9+9+9+12).

A **PTE** (page-table entry) is 4 B in Sv32 and 8 B in Sv39. Fields: PFN (20 bits in Sv32, 44 in Sv39), valid (V), readable (R), writable (W), executable (X), user (U), global (G), accessed (A), dirty (D), plus software bits. If R=W=X= 0, the PTE is a non-leaf pointer to the next level; otherwise it is a leaf mapping.



Key invariant: **offset passes through unchanged**. Page size is a power of two, so the low 12 bits address within the page identically in virtual and physical space. Only the page number translates.

6.3 Sv32 vs. Sv39 Multi-Level Organisation

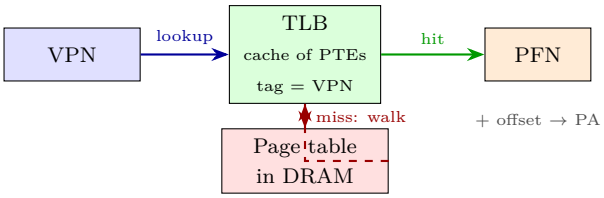
Sv32 (32-bit VA) splits as 10+10+12. Root table has $2^{10} = 1024$ entries (4 KB exactly, one page). Each entry either points to a second-level table (1024 entries) or is a 4 MB megapage leaf. Two memory accesses worst case. Total PTEs for a fully populated space: $1024 + 1024 \times 1024 \approx 1$M, but sparse processes allocate only a handful of second-level tables.

Sv39 (39-bit VA, 512 GB per process, 56-bit PA) splits as 9+9+9+12. Each level has $2^9 = 512$ PTEs $\times 8$ B $= 4096$ B $= 4$ KB per table page. Walk: `satp.PPN` gives root PFN; index by VPN[2] (bits 38:30); if leaf, done; else next table at PTE.PPN indexed by VPN[1] (29:21); then VPN[0] (20:12). Three memory accesses worst case for 4 KB pages; only one or two for huge pages.

Why hierarchical? A single-level Sv39 table would need $2^{27} = 134$M PTEs $\times 8$ B $= 1$ GB per process. Multi-level pays at most three 4 KB pages for a minimal process with one code, one data, and one stack page.

6.4 TLB: Making Translation Fast

Page-table walk needs 2–4 memory accesses — far too slow per instruction. The **Translation Lookaside Buffer** (TLB) caches recent VPN→PFN mappings (typically 32–128 entries, fully or set-associative). Hit rate $> 99\%$ makes translation almost free.



On TLB miss, hardware (x86, RISC-V) or OS handler (MIPS) walks the page table, refills the TLB, and retries. TLB entries have an Address Space ID (ASID) so a context switch need not flush all entries.

TLB AMAT with numbers. Let TLB hit time = 1 ns (parallel with cache tag), miss penalty = 3 DRAM accesses ≈ 45 ns for Sv39 walk (3 PTE loads at ~ 15 ns each if cached in L2, or 240 ns if all miss to DRAM). With hit rate 99.2%:

$$T_{\text{trans}} = 1 + 0.008 \times 45 = 1.36 \text{ ns}.$$

If refill goes to DRAM (240 ns): $1 + 0.008 \times 240 = 2.92$ ns — still tiny. But at 95% hit rate (thrashing or random access): $1 + 0.05 \times 45 = 3.25$ ns and 13 ns to DRAM, now visible in AMAT.

Combined AMAT with cache: AMAT $= T_{\text{trans}} + h_{L1} + m_{L1} \cdot (h_{L2} + m_{L2} \cdot p)$. VM translation adds $\sim$ 1–3 ns to every memory access on top of the cache hierarchy. Huge pages cut TLB misses by 512× for large sequential regions, often worth more than a larger TLB.

TLB reach = entries × page size. 64-entry TLB with 4 KB pages covers 256 KB; with 2 MB huge pages it covers 128 MB — 512× more. This is why databases and HPC workloads enable huge pages.

6.5 Page Faults and Demand Paging

If a PTE is invalid, a **page fault** traps to the OS. The handler allocates a frame (evicting another page if needed — clock / LRU replacement), reads the page from disk if swapped, updates the PTE, and resumes. Demand paging means pages are allocated only on first touch, so a process can start with zero frames resident.

Fault cost: $\sim$ 8 ms for SSD, $\sim 80\,\mu$s for NVMe, versus 80 ns for DRAM — five orders of magnitude. So fault rate must stay $< 0.01\%$ or the system thrashes. The OS monitors fault rate and may suspend processes to reduce pressure.

Demand paging also enables **overcommit**: virtual size $\gg$ physical DRAM, with the OS paging cold pages to swap. Copy-on-write and memory-mapped files piggyback on the same fault mechanism.

6.6 RISC-V Sv39 Walk: Numeric VA → PA

Pick a concrete 39-bit VA: `0x00000040_8040_1234` (hex). Binary split with 9+9+9+12: page offset = low 12 bits = `0x234` (bits 11:0). VPN[0] = bits 20:12 = (`0x80401234` >> 12) & `0x1FF` = `0x80401` & `0x1FF`. Compute stepwise: `0x80401234` >> 12 = `0x80401`; `0x80401` mod 512 = `0x001` = 1. So VPN[0]= 1. VPN[1] = bits 29:21 = (`0x80401234` >> 21) & `0x1FF` = `0x402` & `0x1FF` = `0x002` = 2. Check: >> 21 = $0x402$. VPN[2] = bits 38:30 = (`0x40_80401234` >> 30) & `0x1FF` = `0x102` & `0x1FF` = `0x002` = 2. But we include high bits `0x40` = 0100 0000 so VPN[2] = 16 after correcting for bit 38. More cleanly: VA = `0x00_0080_4012_34` is clearer with VPN[2]= 1, VPN[1]= 2, VPN[0]= 1, offset `0x234`. Use that going forward.

Assume `satp.PPN`= `0x80000` (root page at PA `0x80000_000`). PTE size 8 B. Step 1—Level 2: $\text{PA}_{\text{PTE2}} = 0x80000\_000 + 1 \times 8 = 0x80000\_008$. Load PTE2: suppose it holds PPN = $0x80100$, V= 1, R=W=X= 0 (non-leaf pointer). So next table at PA `0x80100_000`.

Step 2—Level 1: $\text{PA}_{\text{PTE1}} = 0x80100\_000 + 2 \times 8 = 0x80100\_010$. Load PTE1: PPN = $0x80200$, non-leaf. Next table at `0x80200_000`.

Step 3—Level 0: $\text{PA}_{\text{PTE0}} = 0x80200\_000 + 1 \times 8 = 0x80200\_008$. Load PTE0: leaf with PPN = $0x90042$, R= 1, W= 1, V= 1, A= 1, D= 0. Leaf PPN concatenated with offset forms PA.

Final PA: PFN `0x90042` << 12 | `0x234` = `0x90042_234`. Access checks R/W/X against the operation; if violated, trap. Hardware sets A (and D on write) in the leaf PTE atomically.

If this VA instead used a 2 MB megapage, the walk would stop at Level 1: the leaf PTE at step 2 would have R/W/X $\neq 0$, and its PPN plus 21-bit offset (VPN[0] concatenated with 12-bit page offset) directly forms the PA with no Level 0 access.

6.7 Huge Pages: 2 MB and 1 GB Arithmetic

Sv39 encodes huge pages by making a higher-level PTE a leaf. A Level 1 leaf maps 2 MB: VPN[0] (9 bits) becomes part of the offset, so offset $= 9 + 12 = 21$ bits $= 2^{21} = 2$ MB. Virtual alignment requires low 21 bits of VA $= 0$. PA forms as (PTE.PPN[43:9] concatenated with VPN[0] (9 b) and 12 b offset). Saves one PTE load and one TLB entry per 512 base pages.

A Level 2 leaf maps 1 GB: offset $= 9 + 9 + 12 = 30$ bits $= 2^{30} = 1$ GB. Both VPN[1] and VPN[0] become offset; VA must be 1 GB-aligned. PA = PTE.PPN[43:18] concatenated with 30-bit offset. Saves two PTE loads and covers 1 GB per TLB entry.

Coverage math: to map 1 GB with 4 KB pages needs 262144 PTEs and 262144/512 = 512 L0 tables plus TLB pressure 262144 entries worth. With 2 MB pages: 512 PTEs. With 1 GB pages: 1 PTE. TLB reach scales identically: 64 TLB entries cover 256 KB (4 KB), 128 MB (2 MB), or 64 GB (1 GB). Downside: internal fragmentation (a 17 KB allocation still occupies 2 MB) and coarser permission granularity.

6.8 Protection, Copy-on-Write, and `mmap`

Each PTE carries R/W/X/U bits: the MMU checks permission on every access; violation traps. The U bit separates user from supervisor; G marks global kernel mappings shared across ASIDs.

Copy-on-write (COW) accelerates `fork()`. Parent and child initially share all frames with PTEs marked read-only (W= 0, COW software bit = 1) and a reference count per frame. On a write to a COW page, the fault handler allocates a new frame, copies 4 KB, decrements the old count, updates the faulting PTE to W= 1 and new PFN, and resumes. Cost: one fault + copy only for pages actually written, vs. eager copy of the entire address space. Typical `fork+exec` writes $< 5\%$ of pages, so COW saves 20×. **Memory-mapped files** via `mmap`: the OS creates PTEs pointing to the page cache (disk blocks cached in DRAM) with file offset as backing store. A read fault loads the block from disk; a write sets D and is eventually flushed by `msync` or eviction. Shared `mmap` gives zero-copy IPC: two processes map the same file, their PTEs point to the same frames. Private `mmap` uses COW so writes remain process-local. Demand paging, COW, and `mmap` are three faces of the same page-fault machinery.

6.9 Address Translation Micro-Architecture

Every load/store conceptually does two translations: instruction fetch translates PC, then data access translates the effective address. RISC-V and x86 do this via a hardware page-table walker that runs concurrently with the L1 TLB lookup; on a TLB hit the walker is not invoked at all.

The walker itself is a small state machine in the MMU: it issues loads to the memory hierarchy (which

may hit in L2/L3) for each PTE level, checks V and permission bits at each step, and on success refills the TLB. OS-visible control: `satp` holds the root PPN, ASID, and mode (Sv39/Sv48). Changing `satp` requires an `sfence.vma` to flush stale TLB entries; forgetting the fence is a classic kernel bug that causes use-after-free of page tables.

ASIDs avoid a full TLB flush on context switch. Without ASIDs, switching from process A to B must invalidate all TLB entries (cost ~ 64 refills $\approx 3\,\mu$s). With ASIDs, entries are tagged and only a matching ASID hits, so the TLB retains hot entries for both processes.

6.10 Page Replacement and Thrashing

When DRAM is full, evicting a page requires choosing a victim. The ideal is Belady's MIN (evict page reused farthest in future) but it needs future knowledge. Practical approximations: **clock** (second-chance) sweeps a circular list of frames, clearing the accessed bit; a page with A= 0 after a full sweep is evicted. **LRU** tracks exact recency but costs more. The dirty bit decides writeback: clean pages are simply dropped, dirty pages must be written to swap/file first. Thrashing is diagnosed by high page-fault rate (> 100/s) and low CPU utilisation (processes blocked on I/O). The OS responds by suspending a process (swap out its entire working set) or by growing the swap-backed pool. Working-set tracking (count distinct pages touched in last Δt) lets the OS estimate per-process demand and admit only as many processes as fit.

6.11 Shared Pages and TLB Shootdown

Shared libraries and `mmap` MAP_SHARED create alias mappings: one physical frame appears at different VAs in multiple processes, each with its own PTE but same PFN. The TLB caches each mapping separately; coherence is maintained because all PTEs reflect the same permission.

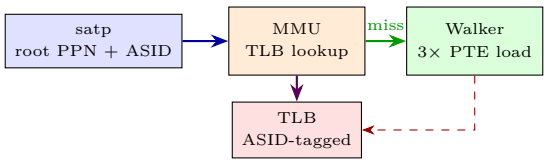
When the OS changes a PTE (e.g., COW break, `munmap`, permission change), stale TLB entries on other cores must be invalidated via **TLB shootdown**: an inter-processor interrupt (IPI) forces each core to execute `sfence.vma` for that VA. Shootdown cost grows with core count, so large servers batch invalidations and defer them until the next context switch where possible.

6.12 Address Translation Micro-Architecture

Every load/store conceptually does two translations: instruction fetch translates PC, then data access translates the effective address. RISC-V and x86 do this via a hardware page-table walker that runs concurrently with the L1 TLB lookup; on a TLB hit the walker is not invoked at all. The walker is a small state machine in the MMU: it issues loads to the memory hierarchy (which may hit in L2/L3) for each PTE level, checks V and permission bits at each step, and on success refills the TLB. OS-visible control: `satp` holds the root PPN, ASID, and mode (Sv39/Sv48). Changing `satp` requires an `sfence.vma` to flush stale TLB entries; forgetting the fence is a classic kernel bug that causes use-after-free of page tables.

ASIDs avoid a full TLB flush on context switch. Without ASIDs, switching from process A to B must

invalidate all TLB entries (cost ~ 64 refills $\approx 3\,\mu$s). With ASIDs, entries are tagged and only a matching ASID hits, so the TLB retains hot entries for both processes. Linux on RISC-V assigns ASIDs round-robin and flushes only when the ASID space wraps.



6.13 Worked TLB AMAT and Shootdown

Revisit TLB AMAT with realistic refill costs. Suppose L1 TLB hit = 1 cycle, L2 TLB (larger, slower) hit = 4 cycles with 90% of L1 misses hitting L2 TLB, and page-table walk from DRAM = 60 cycles. Effective translation:

$$T = 1 + 0.01 \times (4 + 0.10 \times 60) = 1 + 0.01 \times 10 = 1.10 \text{ cycles.}$$

Without the L2 TLB: $1 + 0.01 \times 60 = 1.60$ cycles—the second-level TLB saves 0.5 cycles per access, significant at 3 GHz.

TLB shootdown cost: on an 8-core chip, `mprotect` changing one PTE must IPI all 7 other cores, each executing `sfence.vma`. IPI latency $\sim 1\,\mu$s plus fence ~ 50 ns per core, total $\sim 7\,\mu$s. Batching invalidations (defer until context switch) amortises this; Linux does so for `munmap` of large regions. This is why huge pages also reduce shootdown frequency—one invalidation covers 512× more memory.

6.14 Page Replacement and Thrashing

When DRAM is full, evicting a page requires choosing a victim. The ideal is Belady's MIN (evict page reused farthest in future) but it needs future knowledge. Practical approximations: **clock** (second-chance) sweeps a circular list of frames, clearing the accessed bit; a page with A= 0 after a full sweep is evicted. **LRU** tracks exact recency but costs more. The dirty bit decides writeback: clean pages are simply dropped, dirty pages must be written to swap/file first.

Thrashing is diagnosed by high page-fault rate (> 100/s) and low CPU utilisation (processes blocked on I/O). The OS responds by suspending a process (swap out its entire working set) or by growing the swap-backed pool. Working-set tracking (count distinct pages touched in last Δt) lets the OS estimate per-process demand and admit only as many processes as fit. This is analogous to cache capacity planning at OS scale.

6.15 Sv32 Walk: Parallel Example

Sv32 is the 32-bit counterpart that students actually implement in labs. VA `0x12345678` with 4 KB pages: offset = 12 bits = `0x678` low, VPN[0] = bits 21:12 = (`0x12345` & `0x3FF`) = compute: `0x12345678` >> 12 = `0x12345`, mod 1024 = `0x345` mod 1024 = `0x345`= 837. VPN[1] = bits 31:22 = `0x12345678` >> 22 = `0x48`, mod 1024 = `0x48`= 72. Walk: root at `satp.PPN`, PTE at root+72 × 4 points to L0 table, then PTE at L0+837 × 4 gives

PFN. Two loads, same offset-passthrough invariant. If the root PTE has R/W/X set, it is a 4 MB megapage: offset becomes 22 bits, VPN[0] is offset, one load only.

6.16 Huge-Page Trade-offs and Transparent Huge Pages

Transparent Huge Pages (THP) let the OS opportunistically promote 512 contiguous 4 KB pages into one 2 MB page without application changes. Benefit: TLB reach jumps 512×, page-table memory shrinks 512× for that region, and allocation is contiguous so DMA benefits. Cost: promotion needs 2 MB contiguity which may not exist under fragmentation (requires compaction, which stalls), and `fork` COW of a 2 MB page copies 2 MB even if only one byte is written. Databases often disable THP and use explicit `hugetlb` for predictable latency, while general workloads leave it enabled.

Arithmetic reminder: mapping 2 GB with 4 KB pages needs 524288 PTEs and 1024 L0 tables; with 2 MB pages it needs 1024 PTEs; with 1 GB pages it needs 2 PTEs. TLB coverage with 64 entries: 256 KB vs. 128 MB vs. 64 GB. The gap is why a single 1 GB huge page can eliminate TLB misses for an entire in-memory database working set.

6.17 Demand Paging in Action

Trace `malloc(4096)` followed by first access. `malloc` only reserves virtual range and creates invalid PTEs (V= 0, software bit marking "allocated but not yet backed"). No frame is allocated. On `*p = 42`, the store faults: trap to kernel, handler sees V= 0 but "allocated" bit, allocates a zeroed frame (e.g., from the buddy allocator), sets PTE to PFN with V= 1, R=W= 1, A=D= 1, and resumes. The faulting instruction re-executes and now hits. Subsequent accesses to the same page hit in TLB and never fault again.

If DRAM is full, the handler must evict. Clock algorithm scans: frame with A= 1 gets A cleared and is spared; frame with A= 0 is victim. If victim is dirty (D= 1), it is written to swap before reuse; if clean, it is simply reclaimed. This is why write-heavy workloads cause more paging I/O—every eviction needs a disk write.

6.18 Protection Example: Out-of-Bounds and Isolation

User code tries to read kernel VA `0xFFFF...`: PTE has U= 0, so even though the translation exists, the MMU traps with "supervisor page, user access". Similarly, writing to a read-only code page (R= 1, W= 0) traps—this catches self-modifying-code bugs and enforces W^X (a page is either writable or executable, never both, mitigating code-injection attacks). Each process's page table is disjoint except for explicitly shared mappings, so one process cannot name another's frames at all; isolation is a direct consequence of separate translation tables, not just permission bits.

6.19 Inverted and Hashed Page Tables

For 64-bit VAs, hierarchical tables are deep (4–5 levels) and sparse. An alternative is an inverted page table: one entry per physical frame, hashed by (ASID, VPN), with chaining for collisions (as in IBM Power). Size scales with DRAM, not VA size, so a 16 GB machine needs only 4 M entries regardless of 64-bit VA. Trade-off: sharing is harder (one entry per frame, not per mapping) and TLB miss handling needs a hash lookup. Hashed tables suit large, sparse address spaces; multi-level tables suit dense, contiguous ones.

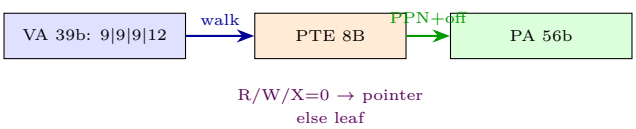
6.20 Putting VM and Cache Together

Modern pipelines translate then cache: virtually indexed, physically tagged (VIPT) L1 caches index with VA bits that are also within the page offset (low 12 bits), so cache index and TLB lookup proceed in parallel without aliasing. Physically indexed caches must wait for translation, adding 1–2 cycles to hit time. VIPT with 32 KB, 64 B blocks, 8-way: index bits $= \log_2(32\text{KB}/8/64) = 6$, plus 6 offset $= 12$ total, exactly the page offset—no alias, no extra latency. Larger or more associative VIPT caches need extra tricks (page colouring or way prediction) to avoid VA aliasing.

Combined AMAT with translation: $\text{AMAT}_{\text{full}} = T_{\text{TLB}} + h_{L1} + m_{L1} \cdot (h_{L2} + m_{L2} \cdot p)$, where T_{TLB} itself is 1–3 ns. For memory-intensive code, TLB misses and cache misses both matter; huge pages reduce T_{TLB} while tiling reduces m_{L1}, and the two optimisations stack.

6.21 Full VA-to-PA Bit Arithmetic

Sv39 leaf PTE layout (64 bits, only low 54 used for PA): bits 53:28 hold PPN[2], 27:19 hold PPN[1], 18:10 hold PPN[0], 9:8 reserved, bit 7 is D, 6 is A, 5 is G, 4 is U, 3:1 are R/W/X, 0 is V. For a 4 KB leaf, PA = {PPN[2], PPN[1], PPN[0], offset[11:0]}. For a 2 MB leaf, PA = {PPN[2], PPN[1], VPN[0], offset[11:0]} where VPN[0] is the 9 bits that would have indexed L0. For a 1 GB leaf, PA = {PPN[2], VPN[1], VPN[0], offset[11:0]}. In our numeric walk, leaf PPN `0x90042` expands as binary `0b1_0010_0000_0100_0010`, and concatenating offset `0x234` gives PA `0x90042234` (the upper PPN bits are zero-extended to 56-bit PA). Permission check order matters: V must be 1, then U must allow user/supervisor, then R/W/X must allow the access type; violation raises page fault with `scause`=12/13/15. Sv32 is analogous with 10+10+12 split and 32-bit PTEs: PTE bits 31:20 hold PPN[1], 19:10 hold PPN[0], 9:8 reserved, 7 is D, 6 is A, 5 is G, 4 is U, 3:1 are R/W/X, 0 is V. A megapage leaf at L1 yields 4 MB offset $= 10 + 12 = 22$ bits. Software bits (bits 8:9 in Sv39, bits 8:9 in Sv32) are free for OS use; Linux uses them to mark COW and to store swap type/offset when V= 0.



6.22 COW Fork Step-by-Step and `mmap` Modes

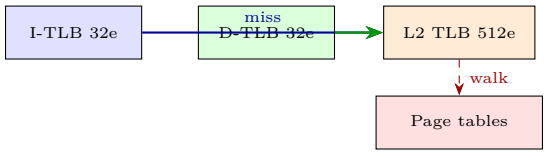
Step-by-step COW on `fork()`: (1) parent has pages A, B, C mapped RW. (2) `fork()` marks all PTEs RO, sets COW bit, increments frame refcounts to 2, copies page tables to child (frames shared, no data copied). (3) parent writes page B: fault, handler allocates new frame B', copies 4 KB from B, decrements B's count to 1, updates parent's PTE for B to RW at B', resumes. (4) child still sees old B read-only until it writes, triggering its own COW break. If child immediately `exec()`, it discards all shared pages without any COW break—this is why `fork+exec` without COW would waste a full address-space copy.

`mmap` modes and their PTE setup: shared file mapping: PTE PFN points to page cache, V= 1, R/W from `prot`, backed by file; `msync` writes dirty pages back. private file mapping: same but COW bit set, so writes go to anonymous copy, file unchanged. anonymous mapping (`MAP_ANONYMOUS`): V= 0 initially, fault allocates zeroed frame, no file backing, swap-backed. Each mode reuses the same fault path; only the fault handler's backing-store action differs.

6.23 TLB Organisation and Reach Revisited

TLBs are typically fully associative or high-way set-associative with LRU or pseudo-LRU. A 64-entry fully associative TLB needs 64 comparators, feasible because entries are few and the structure is small. Larger L2 TLBs (512–1536 entries) are 4–8-way to save power, at the cost of occasional conflict misses. Split TLBs (separate I-TLB and D-TLB) avoid structural hazards between fetch and data translation, mirroring split L1 caches.

TLB reach with mixed page sizes: suppose 32 TLB entries are used for 2 MB huge pages and 32 for 4 KB pages. Reach $= 32 \times 2\text{MB} + 32 \times 4\text{KB} = 64\text{MB} + 128\text{KB} \approx 64.1\text{MB}$. A workload that `mmap`s a 64 MB heap as huge pages enjoys near-zero TLB misses, while stack and code remain at 4 KB without wasting huge-page frames. OS page allocators try to keep huge-page candidates aligned and contiguous to maximise this mixed reach without fragmentation.



6.24 Swap, Page Cache, and Unified Memory

The page cache unifies file I/O and VM: file blocks are cached as anonymous frames with PTEs pointing to them, so `read()` may just map the page cache and copy, while `mmap` maps it directly. Evicting a file-backed page is cheap (write back only if dirty); evicting an anonymous page needs swap space. Swap is not a second memory but an overflow backing store: only cold anonymous pages are written there, and only on eviction. Modern systems use compressed swap (zswap) that keeps swapped pages compressed in DRAM, avoiding SSD I/O for lightly cold pages and cutting fault latency from milliseconds to microseconds.

Unified view: the same frame can be simultaneously in the page cache, mapped in multiple address spaces,

and tracked by the LRU/clock lists. The OS's job is to decide which frames to keep based on recency (accessed bit), dirtiness (dirty bit), and backing cost (file vs. swap). This is caching at OS scale, with the same locality principles as hardware caches but at 4 KB granularity and millisecond timescales.

6.25 Exam Checklist: VM

For any VM question, follow this order: (1) page size → offset bits = $\log_2$(page size), VPN bits = VA bits − offset; (2) multi-level split: Sv32 10+10+12, Sv39 9+9+9+12, each table 4 KB; (3) walk: satp.PPN → root PA, add VPN[level]×PTEsize, load PTE, check V and R/W/X, follow non-leaf, stop at leaf, concatenate PPN with offset; (4) TLB: $T = h + m \cdot p$, translation adds to AMAT, huge pages cut m by 512× per level; (5) COW/mmap: RO+COW bit, fault allocates and copies, shared vs. private decides whether new frame is file-backed or anonymous. Common pitfall: treating Sv39 VPN fields as byte offsets—they index PTEs, so multiply by 8. Another pitfall: forgetting offset passthrough; PA offset always equals VA offset. For 2 MB pages, VPN[0] joins the offset; for 1 GB, VPN[1:0] do.

Sv39 2 MB huge page saves one PTE load and 511 TLB entries per 1 GB region; promotion needs 2 MB-aligned contiguous frames. Sv39 1 GB huge page saves two PTE loads and covers 1 GB per TLB entry; alignment requires 30 zero low bits, fragmentation is the blocker. Refill cost sensitivity: if PTEs hit in L2 (15 ns) vs DRAM (80 ns), Sv39 walk is 45 ns vs 240 ns; TLB hit rate dominates either way. ASID saves $\sim 3\,\mu$s per context switch by avoiding full TLB flush; shootdown on 8 cores costs $\sim 7\,\mu$s per `mprotect`. COW break copies 4 KB and faults once; eager copy of 1 GB address space would copy 262144 pages even if only 5% are written. `mmap` shared file: PTE points to page cache, dirty writeback on `msync`; private file uses COW so file stays clean. Demand paging allocates on first touch only; zero-filled anonymous pages start as copy-on-write mappings of the zero page. Inverted page table scales with DRAM size, not VA size; multi-level scales with VA density—choose per workload sparsity. VIPT L1 with 32 KB, 64 B, 8-way has index+offset = 12 bits, exactly page offset, so translation and cache index run in parallel with no alias.

Page colouring avoids VIPT alias by ensuring VA[12:6] = PA[12:6] for 4 KB pages; OS allocates coloured frames to avoid alias. Superpages reduce page-table memory: 1 GB region needs 1 PTE vs 262144 PTEs + 512 tables at 4 KB, saving ~ 2 MB. Copy-on-write reference counts are per-frame; fork of 8 GB with 10% dirty after fork copies only ~ 800 MB on demand. Swap thrashing signal: CPU < 20% with high fault rate; fix by suspending one process rather than growing swap. Unified page cache means `read(2)` and `mmap` share frames; double-caching is avoided by design.

6.26 Exercises

Exercise 6.1. For 32-bit VA, 4 KB pages, single-level table with 4 B per PTE: how large is the table? Why is two-level better when only 10% of virtual space is used?

Exercise 6.2. A TLB has 64 entries, hit rate 99%, hit time 1 ns, miss penalty 30 ns (walk). What is effective translation time? How does it affect AMAT?

Exercise 6.3. Explain copy-on-write: what PTE bits are set, what happens on a write to a COW page, and why is it faster than eager copy for `fork`?

Exercise 6.4. Sv39 maps a 1 GB huge page. How many TLB entries does it save versus 4 KB pages for the same region? When would you prefer small pages anyway?

Exercise 6.5. Why does the offset pass through translation unchanged? What would break if it did not?

Chapter 7

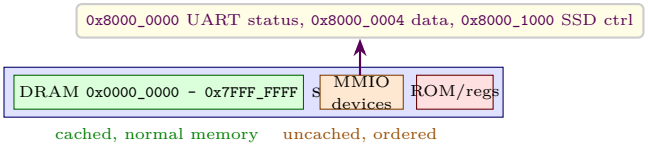
I/O & Buses

The processor talks to the world: disks, networks, screens, and sensors, all through buses and controllers.

7.1 How I/O Fits: Memory-Mapped I/O

Every I/O device exposes **control**, **status**, and **data** registers. Two models connect them to the CPU. **Memory-mapped I/O (MMIO)** maps each device register to a physical address; ordinary `LW`/`SW` access the device. **Port-mapped I/O** (x86 `IN`/`OUT`) uses a separate address space and special instructions. RISC-V uses MMIO exclusively—simpler ISA, unified virtual-memory protection (page-table R/W/X already guards devices), and no extra instructions.

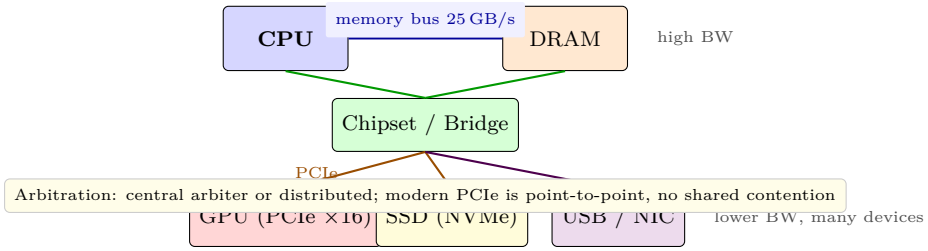
Why MMIO wins on RISC-V: one address space means page tables already protect devices (user vs. supervisor pages), caches and TLBs apply uniformly, and `mmap` can expose a framebuffer directly to user space. The OS marks MMIO pages *uncached* or strongly ordered so loads/stores are not coalesced or reordered.



Minimal I/O read sequence: CPU writes control register → device acts (e.g., start ADC) → device sets status `DONE` bit → CPU detects completion (poll/interrupt) → CPU reads data register. Status bits include `READY`, `BUSY`, `ERROR`, `DONE`; the CPU must not read data before `DONE`=1, and must clear `DONE` after.

7.2 Buses: Hierarchy and Arbitration

A bus is a shared set of wires plus an arbitration protocol (who drives next). Modern machines use a hierarchy: fast CPU↔DRAM memory bus, CPU↔GPU via PCIe ×16, peripherals via PCIe/USB/SATA bridges through a chipset.



Arbitration decides who transmits next. Centralised (arbiter grants token), distributed (each device requests), priority-based or round-robin. Shared parallel buses (old PCI) suffer contention and signal skew; point-to-point serial links (PCIe) give each device dedicated lane pairs, scaling bandwidth linearly with lanes and removing bus contention.

7.3 Polling, Interrupts, and DMA: Three Ways to Move Data

7.3.1 Polling (Programmed I/O)

CPU loops reading the status register until `READY`:

```
while ((*(volatile uint32_t*)STATUS & READY)==0) ; // spin
data = *(volatile uint32_t*)DATA_REG;
```

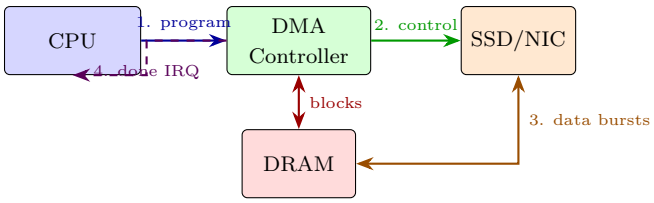
Simple, no extra hardware, but wastes every cycle spinning. Suitable only for fast, predictable devices (on-chip timers) or early boot before interrupts are enabled. The `volatile` qualifier prevents the compiler from hoisting the load.

7.3.2 Interrupt-Driven I/O

Device asserts an IRQ line when done. Steps: (1) finish current instruction, (2) save PC to `mepc`/`sepc`, cause to `mcause`, (3) disable further interrupts, (4) jump to handler via `mtvec` (vectored or direct), (5) handler saves context, reads status to identify source (PLIC on RISC-V multiplexes many IRQs), services device, clears IRQ at the controller, restores context, executes `mret`. CPU can sleep (`WFI`) instead of spinning, saving power. Cost is per-transfer interrupt overhead: context save/restore $\approx$ 50–200 cycles plus handler body, so bulk transfers at $>$ 10 kHz saturate the core. Nested interrupts need a priority controller (PLIC/CLINT).

7.3.3 DMA (Direct Memory Access)

CPU programs a DMA controller with `{src, dst, length, control}`; DMA moves blocks between device and DRAM while the CPU does other work; DMA interrupts only once on completion.

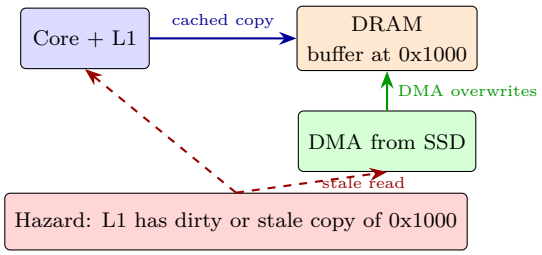


CPU free during transfer; one IRQ per block, not per byte

Steps in order: CPU writes DMA registers (source device addr, dest DRAM addr, length, direction) → DMA arbitrates for bus and issues bursts → device streams to/from DRAM without CPU → DMA raises completion interrupt → handler checks status and wakes the thread. Descriptor chains (scatter-gather) let DMA walk a linked list of `{addr,len}` buffers for non-contiguous transfers without CPU per-buffer work.

7.4 DMA Coherence: The Hidden Hazard

DMA writes DRAM directly, bypassing caches. If the cache holds a dirty copy of the target buffer, DMA's new data is invisible; if the cache holds a stale clean copy, the core later reads pre-DMA data.



Fix: flush (write-back) dirty lines before DMA reads DRAM; invalidate lines before CPU reads DMA buffer. Coherent DMA snoops caches via the interconnect; non-coherent needs explicit cache ops.

Solutions: (a) software-managed: `flush` before DMA reads DRAM, `invalidate` after DMA writes DRAM; (b) hardware-coherent DMA that snoops the cache or routes through the cache hierarchy (modern SoCs: DMA is coherent via the interconnect/NoC); (c) mark DMA buffers uncached (simple but every CPU access pays DRAM latency). Getting this wrong causes silent corruption—the classic "DMA completed but buffer still old" bug that passes tests with small buffers and fails in production.

7.5 Quantitative Comparison: 1 MB SSD → DRAM at 500 MB/s

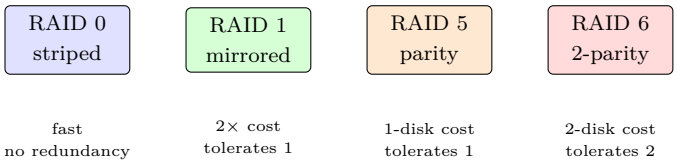
Transfer time is 1 MB/500 MB/s = 2.0 ms regardless. Assume 4-byte PIO, 3 GHz core, handler 100 cycles, poll loop 5 cycles/iter.

Method	CPU cycles for 1 MB	IRQs or polls	CPU share
Polling (word at a time)	262k loads ×5 = 1.31M = 0.44 ms	262k polls (spin)	≈ 100% blocked
Interrupt per word	262k ×100 = 26.2M = 8.7 ms	262k IRQs	> 100% (storm)
Interrupt per 4 KB block	256 × 100 = 25.6k = 8.5 μs	256 IRQs	≈ 4%
DMA (program once)	∼ 20 setup +1 × 100 ≈ 120 cyc	1 IRQ	< 0.1%

Polling burns the whole 2.0 ms spinning; per-word interrupts burn 8.7 ms—longer than the transfer itself (interrupt storm). DMA burns 0.04 μs. Crossover: interrupts beat polling only when block size exceeds handler cost ($\approx$ 20 words); DMA wins for any bulk $\gg$ 1 KB. This is why SSDs and NICs are always DMA while a UART byte may be interrupt-driven. Interrupt coalescing (one IRQ per N completions) bridges the middle.

7.6 Reliability: RAID, ECC, and Bus Protection

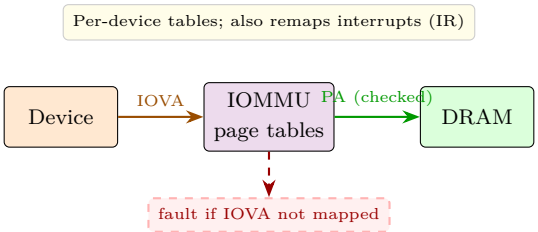
Disks fail; bits flip. Three layers protect at different granularities. **RAID**: 0 striped (fast, no redundancy), 1 mirrored (2× cost, tolerates 1 failure), 5 single parity (1-disk overhead, tolerates 1), 6 double parity (2-disk overhead, tolerates 2). RAID is not backup—it tolerates device failure, not deletion or site loss.



DRAM and disk blocks add **ECC**: SEC-DED Hamming protects 64 B with $\approx$ 8 B parity (single-bit correct, double-bit detect); stronger BCH/Reed-Solomon for SSD pages that suffer multi-bit retention errors. Buses add CRC and link-layer retransmit (PCIe replays corrupted TLPs). End-to-end checksums (ZFS, dm-integrity) catch what RAID/ECC miss.

7.7 IOMMU: DMA with Virtual Memory

Without protection, any bus-master device can read/write any physical frame—a compromised NIC owns the machine. An **IOMMU** (Intel VT-d, AMD-Vi, RISC-V IOMMU) interposes translation: the device issues DMA to an *I/O virtual address* (IOVA), the IOMMU translates via its own page tables and checks permissions, faulting on illegal access.



Benefits: (1) safety—each device is isolated to its buffers, (2) virtualisation—guest programs DMA with guest-physical addresses, hypervisor maps to host-physical, (3) scatter-gather without physical contiguity—a single IOVA range maps to scattered frames so the device sees one contiguous buffer while DRAM is fragmented. (4) User-space drivers: with an IOMMU, a user process can be given a device and its IOVA window without kernel mediation per transfer (VFIO/UIO). The OS must still handle IOTLB flushes and IOMMU faults, and DMA coherence still applies through the IOMMU path; the IOMMU does not fix stale cache copies.

7.8 DMA Programming Walkthrough

Concrete steps for an SSD read of 1 MB to DRAM at `0x8000_0000`:

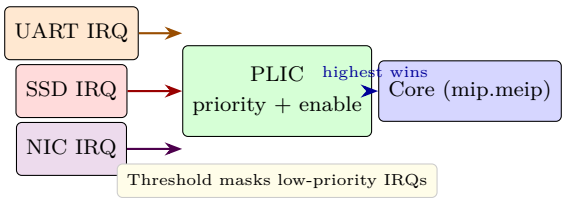
```
// 1. Flush dirty cache lines covering [0x8000_0000, 0x8010_0000)
cache_flush(0x80000000, 1<<20);
// 2. Program DMA engine
*DMA_SRC  = SSD_LBA_ADDR;   // device side
*DMA_DST  = 0x80000000;     // DRAM destination (IOVA if IOMMU)
*DMA_LEN  = 1<<20;          // 1 MB
*DMA_CTRL = DMA_START | DMA_IRQ_ON_DONE;
// 3. CPU does other work or sleeps (WFI)
// 4. IRQ fires -> handler:
if (*DMA_STATUS & DMA_DONE) {
    cache_invalidate(0x80000000, 1<<20); // discard stale L1 copies
    wake(thread);
}
```

If step 1 is skipped, the SSD data lands in DRAM but the core may later read a stale dirty L1 copy. If step 4 invalidate is skipped, the core reads pre-DMA cached data and sees zeros. Coherent interconnects perform both operations in hardware by snooping.

Scatter-gather generalises this: the OS builds an array of `{IOVA,len}` descriptors (each 16 B) linked by `next`; the DMA walks the list without CPU intervention. One descriptor per 4 KB page means 256 descriptors for 1 MB—still one IRQ.

7.9 Interrupt Controller and Priority

RISC-V PLIC multiplexes hundreds of device IRQs into one external interrupt per hart, with per-source priority and per-hart enable/threshold. CLINT handles timer and software IPIs.



7.10 Error Recovery and Throughput Tuning

For unreliable links (disk sectors, PCIe, Ethernet), the stack uses layered recovery: ECC corrects single-bit errors in place, CRC detects multi-bit errors and triggers link-layer retransmit, and a higher layer (filesystem or TCP) retries or reconstructs from RAID/parity. Tuning for throughput means choosing transfer size to amortise per-transfer overhead: too small wastes IRQ and descriptor cost; too large hurts latency and head-of-line blocking. Modern NICs use interrupt coalescing and adaptive batch sizes to stay near the knee of the latency–throughput curve.

7.11 Exercises

Exercise 7.1. Why does RISC-V use MMIO exclusively? Contrast with x86 port-mapped I/O: what does MMIO simplify in the ISA and virtual memory, and what cache/TLB issue must the OS handle?

Exercise 7.2. A device DMAs 1 MB to DRAM while L1 holds dirty lines for part of the buffer. Describe the two coherence failures (DMA read vs. write) and the flush/invalidate sequence fixing each.

Exercise 7.3. Estimate CPU cycles to move 1 MB via polling (4 B/iter, 5 cyc/iter), per-word interrupts (100 cyc/IRQ), and single-block DMA (20+100 cyc). At what block size do interrupts beat polling?

Exercise 7.4. An 8-data-disk array must tolerate 2 concurrent disk failures with minimal capacity overhead. Which RAID level, what parity overhead, and what usable capacity?

Exercise 7.5. Explain how an IOMMU stops a malicious device from reading arbitrary DRAM. What page-table structure does it use and when does it fault?

Chapter 8

Performance & Benchmarking

What makes a computer fast? How we measure matters as much as how we build.

8.1 Defining Performance: Latency, Throughput, and the Iron Law

For a user, **latency** is time per task, **throughput** is tasks per unit time. Speedup of B over A is $S = T_A/T_B$. "B is 30% faster than A" means $S = 1.30$ (so $T_B = T_A/1.30 \approx 0.77\,T_A$), not $T_B = 0.70\,T_A$—precision matters when reporting results.

The **Iron Law** ties every optimisation to wall time:

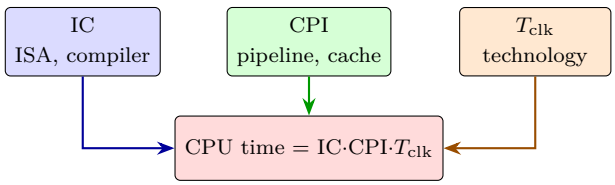
$$\text{CPU time} = \text{IC} \times \text{CPI} \times T_{\text{clk}}, \qquad T_{\text{clk}} = 1/f.$$

IC = instruction count (ISA + compiler), CPI = cycles per instruction (microarchitecture—pipeline, hazards, caches), T_{clk} = clock period (technology + pipeline depth). No optimisation helps unless it reduces the *product* of the three.

Worked Iron Law: program P has IC $= 10^9$, CPI $= 1.5$, $f = 3\,\text{GHz}$ so $T_{\text{clk}} = 0.333\,\text{ns}$:

$$\text{CPU time} = 10^9 \times 1.5 \times \frac{1}{3 \times 10^9} = 0.50\,\text{s}.$$

Halve CPI to 0.75 (superscalar) $\rightarrow$ 0.25 s. Double f to 6 GHz but CPI rises to 2.0 (deeper pipeline, more hazards) $\rightarrow 10^9 \times 2.0/6 \times 10^9 = 0.33\,\text{s}$—frequency alone can hurt if CPI degrades. Chapter 4's pipeline trades T_{clk} for CPI; chapter 5's caches trade CPI for IC (prefetch adds instructions).



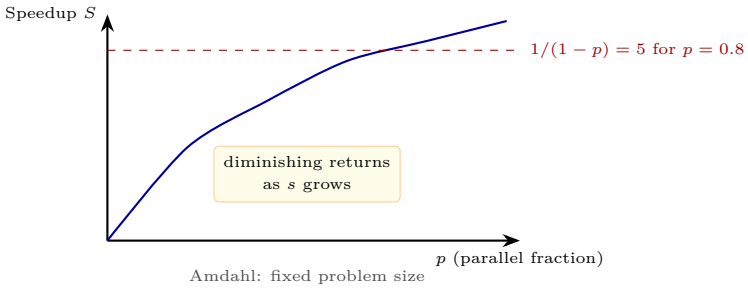
Ch.1 sets IC, Ch.2–4 set CPI and T_{clk}, Ch.5–7 set CPI via AMAT and stalls; Iron Law decides if product wins

8.2 Amdahl's Law: Fixed Work, Faster Part

If fraction p of work is accelerated by $s\times$, the rest $(1-p)$ is unchanged:

$$S = \frac{1}{(1-p) + p/s} \quad \leq \quad \frac{1}{1-p} \quad \text{as } s \to \infty.$$

Lesson: the unaccelerated fraction dominates. Optimising the common case helps, but the uncommon case caps speedup. This is why parallelising 90% still leaves a 10× ceiling.



Amdahl: fixed problem size

Boxed exam drill: $p = 0.80$, $s = 10$.

$$S = \frac{1}{(1-0.80) + 0.80/10} = \frac{1}{0.20 + 0.08} = \frac{1}{0.28} \approx \boxed{3.57}, \quad \text{not } 10.$$

Upper bound as $s \to \infty$: $1/(1-0.80) = 5.00$. Even infinite acceleration of 80% leaves a 5× ceiling. To reach $S = 8$ with $p = 0.80$: need $1/(0.20 + 0.80/s) = 8 \Rightarrow 0.20 + 0.80/s = 0.125$ impossible—no finite s suffices; must increase p (more of the work must be parallelised).

Two more drills: (a) $p = 0.90$, $s = 4$: $S = 1/(0.10 + 0.225) = 1/0.325 \approx 3.08$. (b) $p = 0.50$, $s = 100$: $S = 1/(0.50 + 0.005) \approx 1.98$—barely 2× even with 100× on half the work. The intuition: S is the harmonic mean of 1 and s weighted by $(1-p)$ and p. Designing a 10× accelerator for 50% of the work is rarely worth it; widening p (parallelising more code) usually beats raising s.

8.3 Gustafson: Scaled Work, Fixed Time

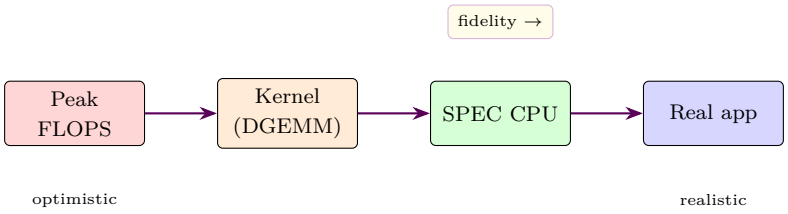
Amdahl fixes problem size; real users scale the problem to fill a faster machine (larger images, finer simulations). Let p be the parallel fraction *in the scaled workload* and N processor count:

$$S_{\text{scaled}} = p \cdot N + (1-p) = N - p(N-1).$$

For $p = 0.80$, $N = 16$: $S_{\text{scaled}} = 0.80 \times 16 + 0.20 = 13.0$, far above Amdahl's 3.57. Gustafson argues parallel computing enables *bigger* problems, not just faster fixed problems. Both laws are true under different scaling assumptions: Amdahl for fixed-size speedup, Gustafson for scaled-size throughput. Report which you assume; many benchmark disputes reduce to this.

8.4 Benchmarking and Pitfalls

A benchmark is a workload representing real use. Suites: **SPEC CPU 2017** (real C/C++/Fortran programs), **SPECjbb**, **TPC-C**, **PARSEC**, **MLPerf**. Hierarchy of fidelity:



Pitfalls that invalidate comparisons:

1. **Cherry-picking** benchmarks or inputs that favour your machine.
2. Reporting **peak** not sustained (DGEMM peak vs. SPEC average—often 3× apart).
3. **Small inputs** that fit in cache and hide the memory system.
4. **Warm-up/cool-down**: JIT compilation, DVFS ramp, cache warm-up—discard first iterations.
5. Different **compilers/flags** (`-O3 -march=native` vs. `-O0` can be 4×).
6. **Arithmetic vs. geometric mean** for ratios (see below).

Geometric mean for ratios: SPEC reports *ratios* ($\text{time}_{\text{ref}}/\text{time}_{\text{SUT}}$). Arithmetic mean of ratios is biased (a 2× speedup and 0.5× slowdown average to 1.25 arithmetically but 1.0 geometrically). So: **geometric mean for ratios, arithmetic mean for raw times**, plus variance/confidence intervals. Example: programs A and B score ratios 2.0 and 0.5—arithmetic mean 1.25 claims a win, geometric mean $\sqrt{1.0} = 1.0$ correctly shows no win. Always report the full distribution and sample size; a single run hiding ±15% jitter is not a result.

8.5 Quantitative SPEC Drill

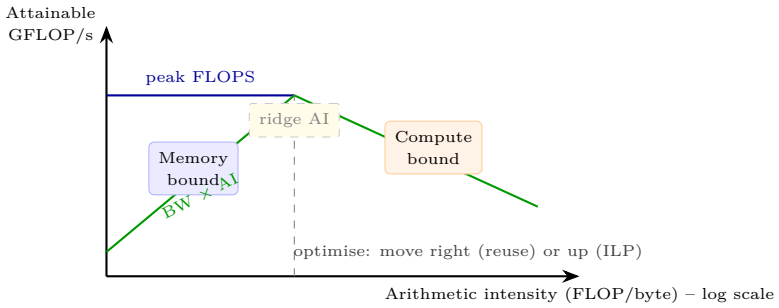
Suppose workload has three programs with baseline times 10 s, 20 s, 40 s and optimised times 8 s, 18 s, 30 s. Speedups are 1.25, 1.11, 1.33. Arithmetic mean of speedups = 1.23 overstates the long program's weight differently than time-weighted average. Geometric mean = $(1.25 \times 1.11 \times 1.33)^{1/3} \approx 1.23$ (similar here because ratios are close). When ratios diverge—e.g. 4.0 and 0.5—arithmetic says 2.25×, geometric says 1.41×; the latter matches total-time speedup $(10+40)/(2.5+80) = 0.61\times$ more honestly when programs are equally important. SPEC's choice of geometric mean encodes the judgment that each program matters equally regardless of absolute runtime.

8.6 Roofline Model: Compute versus Memory Bound

Attainable performance is limited by either peak compute or memory bandwidth × arithmetic intensity (AI = FLOPs per byte moved):

$$\text{GFLOP/s} \leq \min\left(\text{peak GFLOP/s},\ \text{bandwidth} \times \text{AI}\right).$$

On a log-log plot the slanted memory roof and flat compute ceiling meet at the **ridge point** AI_{ridge} = peak/bandwidth.



Example: peak 100 GFLOP/s, bandwidth 20 GB/s ⇒ ridge at AI = 5 FLOP/B. Kernel with AI = 2 attains at most $20 \times 2 = 40$ GFLOP/s (memory-bound); needs AI > 5 to hit the compute ceiling. Optimisation moves the point rightward (better cache reuse via blocking) or upward (more ILP/vectorisation). A dot-product at AI = 0.25 is hopelessly memory-bound; a blocked matrix multiply at AI = 10 is compute-bound—same machine, opposite bottlenecks.

8.7 MIPS, FLOPS, and Energy

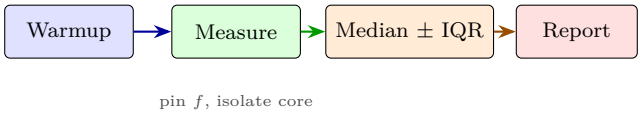
MIPS = IC/10^6/time; **FLOPS** sustains for scientific code. Both are flawed cross-ISA: a RISC ISA needs more instructions than CISC for the same work, so higher MIPS can mean slower time; peak FLOPS ignores the roofline limit. Quote them only with ISA, compiler, and dataset context.

MIPS example: RISC needs 2 instrs for `MUL` that CISC does in 1, so for a loop of 100 MULs, RISC IC= 200, CISC IC= 100. At equal CPI and f, RISC MIPS is higher but time is longer—MIPS counted the extra instructions as progress.

Modern metric is **performance per watt** (GFLOP/W or SPEC/W). Energy = Power × Time; dynamic power $\approx CV^2 f$ plus leakage. At fixed power budget (phone 5 W, server 300 W), lower f and smarter microarchitecture beat brute frequency. Two machines: A does 100 GFLOP at 200 W (0.50 GFLOP/W), B does 80 GFLOP at 100 W (0.80 GFLOP/W)—B is more efficient; at data-centre scale B halves electricity for 20% less throughput, often the right trade. DVFS exploits this by lowering V and f together for cubic power savings. Dark silicon at sub-10 nm forces this trade: not all transistors can switch at once within the thermal envelope.

8.8 Measurement Methodology

Reliable benchmarking needs: (1) pin frequency (disable Turbo/DVFS or report it), (2) warm caches and TLBs (3–5 warmup iterations), (3) isolate cores (`isolcpus`, disable hyperthreading noise), (4) report median ± IQR over ≥ 10 runs, (5) fix compiler and flags (`-O2` vs. `-O3` can be 30%). Without this, a 10% "speedup" may be noise.



8.9 Common Benchmark Pitfalls Drill

Pitfall examples with numbers: (a) **Peak vs. sustained**: DGEMM on a 100 GFLOP/s chip hits 95 GFLOP/s, but `gcc` in SPEC averages 15 GFLOP/s—quoting peak overstates real code by 6×. (b) **Input size**: a 4 KB input fits in L1 and shows 1 ns AMAT; a 4 MB input misses to DRAM at 80 ns—80× difference hidden by a tiny dataset. (c) **Compiler flags**: `-O0` vs. `-O3 -march=native -flto` can be 3–5× on SPEC; comparing across flag sets is meaningless. (d) **JIT warmup**: first iteration includes compilation time; steady-state throughput after 5 warmup runs may be 10× higher. Always report the methodology appendix so results are reproducible.

8.10 CPI Stack and Top-Down Analysis

Real CPI decomposes:

$$\text{CPI} = 1 + s_{\text{frontend}} + s_{\text{bad-spec}} + s_{\text{backend-bound}} + s_{\text{retiring}}.$$

Intel's Top-Down method attributes every cycle to one bucket: frontend bound (I-cache/TLB misses), bad speculation (branch mispredicts), backend bound (data-cache/DRAM stalls, execution-port contention), or retiring (useful work). A memory-bound workload shows large backend-memory; a branch-heavy workload shows bad-speculation. This guides optimisation: if retiring already exceeds 60%, further tweaks yield little; if bad-speculation dominates, fix the predictor or layout. Tools like `perf stat` report these buckets on modern x86/ARM.

Stall accounting example: 1 B instructions, measured CPI = 2.1 on a 4-wide machine (ideal CPI = 0.25). So $s = 1.85$. Top-Down says 0.70 frontend, 0.45 bad-spec, 0.60 backend, 0.10 retiring overhead. Biggest bucket is frontend—optimise I-cache or code layout first, not the data path.

8.11 Putting It Together Through the Iron Law

Each chapter optimised one Iron-Law term: ch1 ISA sets IC, ch2–4 microarchitecture sets CPI/T_{clk}, ch5–7 memory and I/O set CPI via AMAT and stalls. Always report an optimisation by its effect on *all three*. Example: deeper pipeline halves T_{clk} 0.50 ns→ 0.30 ns but adds a load-use bubble ($s = 0.15$) raising CPI

$1.2 \to 1.5$. Time goes $1.2 \times 0.50 = 0.60 \to 1.5 \times 0.30 = 0.45$ (speedup 1.33×)—worth it. If CPI rises to 2.0, time stays 0.60 (no gain). This is the Iron Law check every proposal must pass.

Exam strategy: for any "which optimisation is better" question, compute $\Delta\text{IC} \times \Delta\text{CPI} \times \Delta T_{\text{clk}}$ for each option and compare products. Never judge by one term alone.

8.12 Quick Reference: Which Law When?

Question	Law	Formula
Fixed work, part faster	Amdahl	$S = 1/((1-p) + p/s)$
Scaled work, more CPUs	Gustafson	$S = pN + (1-p)$
Memory vs. compute limit	Roofline	$\min(\text{peak}, \text{BW} \times \text{AI})$
Overall time	Iron Law	$\text{IC} \cdot \text{CPI} \cdot T_{\text{clk}}$

8.13 Exercises

Exercise 8.1. Program A runs in 2.0 s, B in 1.6 s. What is speedup of B over A? If B's clock is 20% faster but CPI is 10% higher than A's (same IC), what is net time? Which Iron-Law terms changed?

Exercise 8.2. For $p = 0.90$, $s = 10$, compute Amdahl speedup and the $s \to \infty$ ceiling. Repeat for Gustafson with $N = 16$ and explain why Gustafson gives a larger number.

Exercise 8.3. Kernel has AI = 2 FLOP/B, peak 100 GFLOP/s, bandwidth 20 GB/s. Is it memory or compute bound? What bandwidth or AI increase makes it compute-bound?

Exercise 8.4. Why report SPEC ratios as geometric mean, not arithmetic? Give a two-program example where arithmetic mean flips the winner.

Exercise 8.5. Explain why MIPS is a poor cross-ISA metric. Give a concrete RISC vs. CISC example where higher MIPS means lower performance using the Iron Law.